



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА
ФАКУЛТЕТ ПО ИЗЧИСЛИТЕЛНА ТЕХНИКА И АВТОМАТИЗАЦИЯ

Катедра
„Софтуерни и Интернет
технологии”

Дипломна работа

Тема:

Проектиране и реализация на уеб базирана
музикална платформа

Изготвил:
Калоян Попов
Специалност: СИТ
Факултетен номер: 21621527

Ръководител:.....
гл.ас. д-р Даниела Петрова

Съдържание

Глава I – Увод	4
Глава II – Анализ на проблема и избор на технологии	6
1. Анализ на проблема	6
2. Обзор на съществуващите решения	7
3. Цели на задачата	11
4. Задачи за разработка на дипломната работа.....	11
5. Избрани технологии.....	12
Глава III. Проектиране на системата	17
1. Изисквания на програмната система.....	17
2. Архитектура на системата.....	19
3. Логически модел	21
4. Диаграма на работния процес	26
5. Организация на данните	27
6. Макет на платформата	33
Глава IV - Реализация на системата	36
1. Структура на данните	36
1.1. Нормализация на таблиците от БД.....	36
1.2. Физически модел и описание на таблиците от БД.....	36
2. Описание на програмните модули.....	48
3. Структура и организация на потребителския интерфейс	55
3.1. Начална страница	55
3.2. Страница за регистрация и вход	58
3.3. Страници за изпълнители и албуми	61
3.4. Страница за профил	62
3.5. Страница за приятели	65
3.6. Страница за любими песни	66
3.6. Страница за предложения	66
3.7. Страница за плейлисти	67
3.8. Страница за администратор	70
4. Responsive дизайн.....	75
Глава V – Тестове и резултати	78
Глава VI – Заключение	78

Глава I – Увод

Разработената web платформа в настоящата дипломна работа дава възможност на потребителите да получават музикално съдържание, благодарение на функционалностите, които са имплементирани в нея, под формата на различни изпълнители, албуми и песни. Методът избран за предаването му е един от най-използваните методи за консумиране на музикално съдържание, а именно чрез аудио възпроизвеждане. По този начин потребителите имат възможност, както да слушат харесваната от тях музика, така и да експериментират с нови и различни жанрове. Разглеждат се и различните функционалности за повишаване на качеството на платформата като съвместни плейлисти между приятели, споделяне на музикални открития и персонализирани препоръки.

Основният акцент в системата е поставен върху изграждането на музикална среда, в която потребителите могат да откриват нови изпълнители, както и да изграждат общи музикални преживявания със своите приятели. Благодарение на съвместните плейлисти и алгоритми за препоръки, платформата дава възможност на хора с общи интереси да се свържат чрез музиката.

Поради нарастващото търсене на дигитални услуги проектът има за цел да бъде максимално удобен за използване, както на мобилни устройства, така и на лаптопи, настолни компютри и т.н. Удобният и интуитивен интерфейс е съобразен с нуждите на младата аудитория, привикнала към съвременните технологии, но освен към тях е обърнато и внимание към по-зрялата аудитория, търсеща по-достъпен и лесен начин за слушане и откриване на музикално съдържание.

Повечето платформи за възпроизвеждане на музика не са достъпни на български език, което може да създава бариера за по-широката аудитория в България. Взимайки това под внимание платформата е насочена към българското общество.

Проектът дава възможност на потребителите да използват платформата, без нуждата от създаване на потребителски профил, а именно като гост, което не го прави задължаващо. Престоят им в платформата се реализира по същия начин, както и на регистриран потребител, без лимитации спрямо слушането на музика, но с малки ограничения относно добавянето на любими песни и персонализирани предложения.

Една от по-интересните функционалности в настоящата дипломна работа са плейлистите. Те облекчават потребителите от това да запомнят всяка една от песните, които искат да слушат. С цел да се избегне постоянното ръчно търсене, плейлистите дават възможност да се добавят песни в тях, дори те да не са обозначени като харесани от потребителя. В следствие на това, той може да създава плейлисти от песни, които са фокусирани върху някой конкретен жанр или изпълнител,

позволявайки му да филтрира песните. Плейлистите имат по един създател, но може да имат няколко редактори, добавени от него. При случай, в който потребителят не желае да споделя плейлиста с останалите потребители, той може да зададе статуса на плейлиста като частен, което позволява само на него да го преглежда и редактира.

Глава II – Анализ на проблема и избор на технологии

1. Анализ на проблема

Целта на приложението е потребители от всякаква възрастова група и с технически умения от ниско до високо ниво да имат възможност да го използват. Достъпът до музикално съдържание трябва да бъде масово достъпен чрез интерактивен интерфейс и responsive дизайн. Основната функционалност, която се фокусира върху възпроизвеждането на музикално съдържание, трябва да съществува под различни форми - локализация на песни чрез артисти, албуми и други, създаване на персонализирани плейлисти и система за препоръчване на музика на база потребителски интереси.

Преди да се решава как ще се осъществи самата реализация на проекта, трябва да се избере платформа, на която той да се инициализира. Най-добрият избор за платформа е именно уеб платформата, тъй като така мащабът на обхванатата аудитория от потребители ще бъде най-голям. С дизайна, който отговаря на стандартите за мобилни устройства както и на едно уеб приложение, платформата ще може да бъде освен добре изглеждаща, така и удобна за използване.

Потребителският интерфейс на приложението трябва да бъде представен на български език, тъй като ще бъде употребяван главно на българския пазар и това ще предостави възможно най-интуитивно и лесно разбиране на функционалностите му. По-интуитивният дизайн ще доведе до повторно посещение на уеб платформата от страна на потребителите.

Основният проблем и задача, която решава представената система, е ограничените възможности за откриване на музика и липсата на персонализирано потребителско изживяване.

Стъпките за решаването на тази задача следват с това да се конструират различните функционалности, които ще изпълнява системата. Благодарение на началната идея на различните нужни функции за реализация, може да се създаде нужната база данни, която ще служи за съхранение на информацията. За улеснена работа с уеб приложението всички операции засягащи базата данни ще се осъществяват през самото уеб приложение.

За реализирането на функциите на административна част от базата данни следва да се направи администраторски панел, където администраторът може да извършва нужните операции свързани с базата данни. Също така, за негово удобство, следва да се добавят статистики за потребителите на приложението, което ще подпомага

бъдещи решения по разрастване на платформата и имплементиране на нови функционалности.

Важна част от времето, което потребителите прекарват в употреба на приложението е то да не спира да функционира при определена или неопределена грешка. За да не се стига до принудително спиране, трябва точно да се идентифицират проблемите, които може да възникнат, когато приложението е вече на пазара. Това изисква ясна представа за функциите, които системата ще предоставя, както и подробно обработване на всички възможни случаи на употреба.

Уеб платформата трябва да функционира, както и в режим на логнат потребител, така и без да има нужда потребителя да си създава акаунт. Трябва да се изписват конкретните съобщения, когато потребителят иска да извърши действие, което е достъпно само за потребител със създаден акаунт, който е и логнат по време на изпълнение на действието.

2. Обзор на съществуващите решения

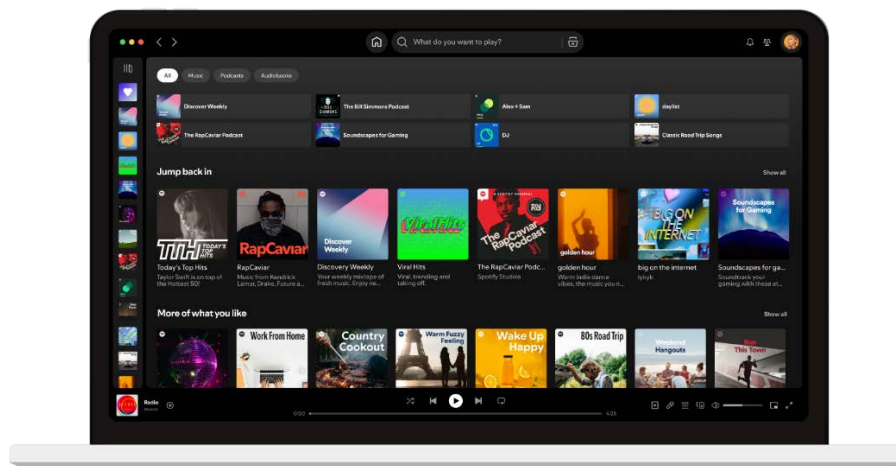
Най-често срещан проблем при музикалните приложения е, че услугите, които предлагат са най-често строго лимитирани относно свободата на музиката, която може да се слуша. Това се изразява в наличието на много реклами по време на употребата на приложението, а в някои случаи и изцяло забраняването на достъпа на потребителя до музикалното съдържание. Много от ограниченията също предотвратяват потребителя от това да използва цели функционалности.

Много често ако функцията на работа на системата изисква двама или повече потребители да взаимодействат, се изисква и от двамата потребители, да закупят месечен или годишен абонамент, за да може да използват услугата.

Друг основен проблем на тези приложения е, че не са ексклузивно достъпни на български език, което може да доведе до загуба на голяма част от българското общество като клиенти на това приложение.

• Spotify

Spotify е една от най-популярните стрийминг платформи за музика, която предлага богат каталог от песни, подкасти и персонализирани плейлисти. Платформата залага на силна персонализация чрез алгоритми и е достъпна на почти всички устройства.



Фигура 1 Начална страница на Spotify

Предимства:

- **Богат каталог с музикално съдържание** - Spotify предлага десетки милиони песни, подкасти и музика от целия свят. Платформата обхваща почти всеки жанр, култура и език.
- **Интелигентна персонализация** - Алгоритмите на Spotify например „Седмични открития“ предлагат нова музика, базирана на скорошната музиката, която е слушал потребителя.
- **Интуитивен интерфейс** - Лесна навигация, възможност за създаване на собствени плейлисти, социални функции и други.
- **Безплатна версия** - Потребителите могат да ползват Spotify безплатно, което го прави достъпен за масова аудитория, въпреки ограниченията.
- **Интеграция с други услуги** - Spotify се синхронизира с Instagram, Discord, Google Maps и други приложения.

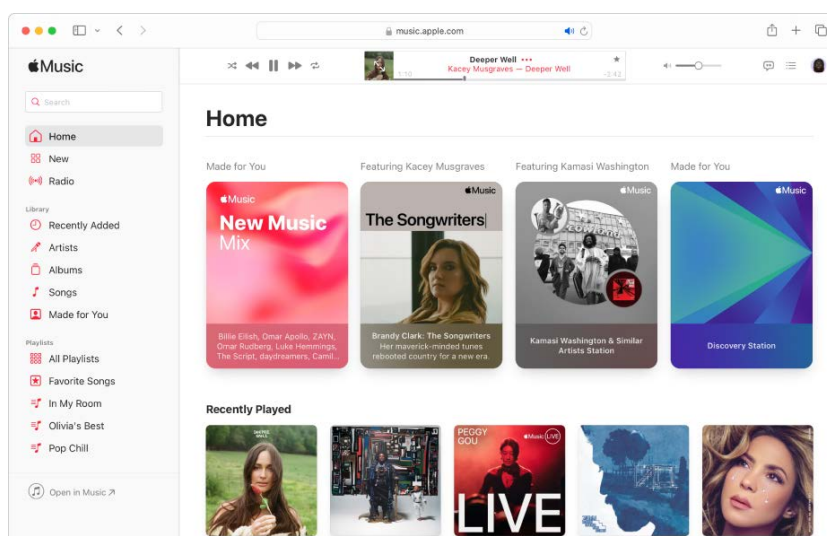
Недостатъци:

- **Реклами в безплатната версия** - Прекъсванията от реклами могат да бъдат неприятни за потребителите, които не желаят да закупят месечен абонамент, което може да доведе до спад на употребата на приложението.
- **Ограничен офлайн достъп** – Само потребители, които са закупили месечен абонамент могат да се възползват от услугата за офлайн слушане.
- **Други ограничения свързани с безплатната версия** – В безплатната версия на приложението потребителите имат ограничен брой пропускания

на песните, което може да доведе до голямо недоволство от тяхна страна и изцяло прекратяване на използването на приложението.

- **Apple Music**

Apple Music е музикална платформа, създадена от Apple, която предлага богат каталог от музика и ексклузивно съдържание. Платформата е силно интегрирана с екосистемата на Apple и акцентира върху високо качество на звука. За да може потребителите да се възползват максимално и най-лесно от тази платформа те трябва да притежават устройство, произведено от Apple. Въпреки това Apple Music приложението е достъпно и на Android и Windows устройства.



Фигура 2 Начална страница на Apple Music

Предимства:

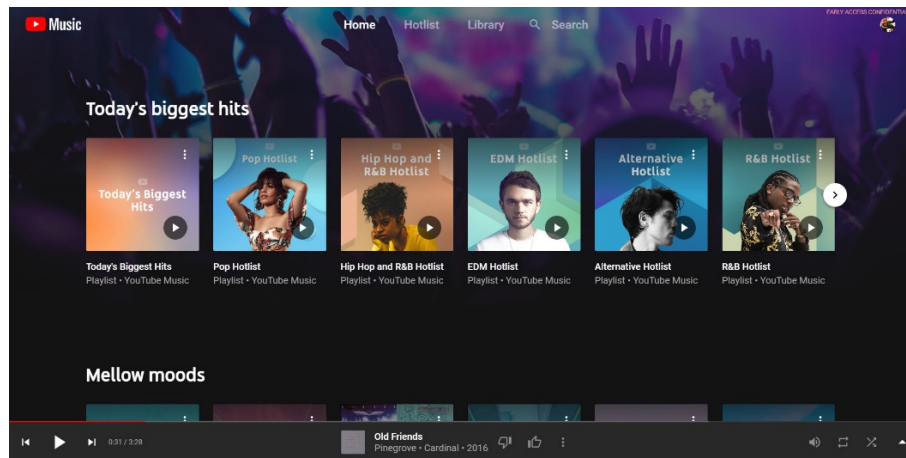
- **Hi-Fi аудио и Dolby Atmos** - Поддръжката на lossless аудио и качественото предаване на звук прави слушането по-стойностно и ангажиращо.
- **Интеграция с Apple екосистемата** - Перфектно съвместима с iPhone, iPad, Mac, Apple Watch, CarPlay и Siri, предоставяйки улеснен начин на употреба за потребителите.
- **Ексклузивно съдържание** – Много често голяма част от изпълнителите, пускат ексклузивно съдържание налично първо за потребителите на Apple Music.
- **Отделен раздел за класическа музика** - Приложението *Apple Music Classical* предлага специализирана база за любителите на класическата музика.

Недостатъци:

- **Няма безплатна версия** – Apple Music потребителите няма достъп без абонамент, което ограничава безплатния достъп, освен за бърз преглед на песните, което не е достатъчно за качествено прекарано време в платформата, освен ако потребителят не заплати месечен абонамент.
- **По-тромав интерфейс** – В сравнение с други конкурентни платформи, платформата на Apple Music може да се счете за неинтуитивна и сложна за навигиране.
- **По-слабо изграден алгоритъм за препоръки** - Персонализацията не е толкова усъвършенствана, както и начините за препоръки на музикално съдържание на потребителите.

- **Youtube Music Premium**

YouTube Music Premium е музикалната услуга на Google, която комбинира аудио и видео съдържание в една платформа. Платформата използва предимствата на YouTube за достъп до базата си от данни за музикални видеа, изпълнители, изпълнения на живо и др.



Фигура 3 Начална страница на Youtube Music Premium

Предимства:

- **Голямо разнообразие от съдържание** - Освен официални песни и видеоклипове, предлага ремикси, неофициални записи, концерти и др.
- **Интеграция с YouTube** - Историята на гледане в YouTube помага за изграждането на персонализация спрямо музикалните предпочитания на потребителите.

- **Офлайн слушане, без реклами** - Платената версия предлага офлайн достъп и премахване на рекламите.

- **Фоново слушане** – Youtube Music Premium позволява на потребителите да слушат музика във фонов режим на мобилни устройства, за разлика от неплатената версия на Youtube приложението.

Недостатъци:

- **Силно ограничена безплатна версия** – Без платената версия на приложението, много от функциите са ограничени, като фоновото слушане и офлайн, както и наличието на реклами

- **По-малко развити плейлисти и кураторски функции** – Спрямо конкурентните платформи организацията на плейлистите е по-некачествена.

- **Интерфейсът може да бъде претрупан** - Комбинирането на видео и аудио понякога създават усещане за обърканост и липса на яснота спрямо интерфейса.

3. Цели на задачата

Целта на разработената дипломна работа е да се направи музикална, уеб-базирана платформа, където всякакви потребители да могат да получават музикално съдържание и да търсят такова под различни форми – изпълнители, албуми, жанрове, песни и други. Функционалностите трябва да бъдат правилно реализирани и взаимодействието с базата трябва също да бъде осъществена, чрез интуитивен и лесен за употреба потребителски интерфейс.

4. Задачи за разработка на дипломната работа

Системата трябва да се реализира с определен брой задачи, които са:

- **Възможност за автентикация на потребителите** – Всеки от потребителите трябва да има възможност да има акаунт, който може да използва по време на употребата на приложението. Ако потребителят няма акаунт може да си създаде такъв чрез регистрационна форма. Всички данни въведени от потребителя трябва се валидират и да се изписват съответните съобщения при грешно подадена информация.

- **Създаване на база данни с нужната информация за системата** – Реализацията на базата данни трябва да бъде пряко свързана с разработеното уеб приложение и неговите функционалности, като се обърне достатъчно внимание, за да може всяко взаимодействие/корекция в базата да се обработи правилно.

- **Търсене на музикално съдържание (песни, албуми, изпълнители)** – Трябва да се създаде система, в която да може да се локализира музикално съдържание при нужда от потребителите. Тя трябва да предлага различни начини, по които потребителите могат да откриват музика по лесен и достъпен начин.

- **Възпроизвеждане на музикално съдържание в реално време** – Трябва да се създаде функционалност за плейър, който да визуализира песните. Когато потребителят избере песен за слушане, тя трябва да се пусне на фонов режим, да се визуализира в музикалния плейър и да не прекъсва независимо от в коя част на платформата е потребителят и при промяна на песента тя да се смени с тази, която е избрана.

- **Създаване и управление на плейлисти с песни** – Трябва да се поддържа функция, която ще позволява добавянето на песните в плейлисти, в зависимост от определени ограничения, свързани с това дали потребителят е създател на плейлиста, дали го следва или може да го редактира.

- **Препоръки базирани на предпочитанията на всеки потребител** – Трябва да се следи музикалното съдържание, което потребителите консумират и на база на това да се създаде опция за тях да получават препоръки с песни за определени жанрове.

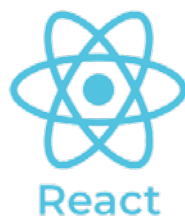
- **Интерактивен потребителски интерфейс с елегантен и естетичен дизайн** - Дизайнът трябва да бъде responsive и да позволява на всякакви устройства да могат да го достъпят без проблем.

- **Отделен административен панел, за въвеждане и коригиране на информация** – Трябва да се създаде администраторски панел, който да работи пряко с базата данни, където може да се извършват различни операции – създаване, редактиране, актуализиране, изтриване, като също така трябва да има и налични статистики за употребата на платформата от потребителите.

5. Избрани технологии

- **React**

React (React.js или ReactJS) е JavaScript библиотека за изграждане на потребителски интерфейс, с която може да се програмират уебсайтове и мобилни софтуерни приложения, която е с отворен код. Поддържа се от Facebook и общност от разработчици и компании.



Фигура 4 React – Лого

React също така се използва и много в изграждането на мобилни приложения, ползвайки "React Native". Библиотеката се ползва в изключително голям брой уебсайтове (над 1 милион) от март 2013-та насам. React налага най-новите JavaScript стандарти (ES6+), което прави изграждането на една апликация по-приятно.

Библиотеката е лесна за навлизане и има едни от най-добрите документации, които да се следват за придобиването на нужните умения. Също така е доста гъвкава. Основата на ReactJS е библиотека за компоненти, оттам нататък разработчиците избират какви пакети да използват за рутиране, валидиране на формуляри, изпращане на HTTP заявки, менажиране на състоянието (Redux, Flux, MobX) и други.

Едно от основните предимства на React, в сравнение с други платформи като Angular, е така нареченото виртуално DOM дърво (или Virtual DOM). Концепция, която дава възможността на изграденият UI да се пази в паметта и да се синхронизира с истинското DOM дърво. Това е огромен плюс при оптимизация на едно приложение.

- **Node.js**

Node.js (или само Node) е сървърно приложение, създаващо среда за изпълняване на JavaScript код (runtime environment), а не език за програмиране. Node.js прави възможно изпълнението на JavaScript код директно на сървъра, както обикновено това се случва в уеб браузъра. Когато се каже, че дадено приложение е написано на Node.js, се разбира, че то е написано на JavaScript и се изпълнява от Node.js.



Фигура 5 Node.js – Лого

Node.js често може да се види използван за високоскоростни приложения, които работят в реално време. Големи компании използват Node.js и го включват като част от дадено голямо приложение или услуга.

Node може да се използва като се инсталира на сървър или на личен компютър. Най-често се инсталира на сървър, за да може създадените за него приложения да са достъпни в уеб, например на собствен виртуален сървър. Node.js може да работи с всякакви сървърни технологии като уеб сървърите Apache, Nginx и различните типове бази данни NoSQL (MongoDB, Redis) и SQL (MySQL) и много други, като предлага и богат избор от технологии и инструменти.

Стандартно езикът за програмиране JavaScript се използва основно за front-end разработка (частта от приложението, с която потребителят взаимодейства) в сайтовете, за да добави някаква интерактивна функционалност и раздвижване на статичното HTML съдържание. JavaScript кодът за front-end частта на сайта/уеб приложението се изпълнява в уеб браузър, с помощта на специален JavaScript интерпретатор/двигател (JavaScript engine).

- **HTML/Tailwind CSS**

HTML (HyperText Markup Language) е основният език за създаване на уеб страници и приложения. Той е стандарт за интернет, който се определя от W3C (World Wide Web Consortium) и се използва за структуриране на съдържанието на уеб страницата чрез използването на тагове и атрибути. HTML е език за маркиране, което означава, че не е език за програмиране, а по-скоро е начин за обозначаване на части от текста, които да бъдат третирани по определен начин от уеб браузър.



Фигура 6 HTML – Лого

Когато някой кликне на линк в страница, или въведе нов URL в адресното поле, браузърът изпраща заявка за документ от Web server-а, където е качена страницата, посредством HyperText Transport Protocol (HTTP). Сървърът след това изпраща документа обратно до потребителя като се визуализира на дисплея от браузър. Всички неща, които са на този документ – текст, снимки, аудио или видео файлове, всички те са подредени посредством HTML структура.

HTML5 е осъвременена версия на едноименния език. Подобренията на HTML5 спрямо предходните версии са много, но най-ключовите от тях са:

- Подобен код – HTML5 предлага на уеб дизайнерите да използват по-чист, по-хубав код. Вече може да се премахнат `<div>` таговете и да се заменят със семантични HTML5 елементи.
- По-елегантни формуляри – опциите за полета за въвеждане на потребителски данни са много по-гъвкави в сравнение с преди.



Фигура 7 Tailwind CSS – Лого

Tailwind CSS е CSS рамка с отворен код. Тя предоставя списък от utility CSS класове, които могат да се използват за стилизиране на всеки елемент чрез смесване и съвпадение.

Tailwind CSS предлага прост и ясен синтаксис за използване. Той се основава на набор от класове, всеки от които съответства на конкретно правило за стил. Това прави процеса на разработка по-прозрачен и разбираем и улеснява бързото внедряване на проектите.

Tailwind CSS също така предоставя огромни инструменти за прилагане на адаптивен дизайн. Може лесно да се дефинира различни класове за различни размери на екрана, което позволява на сайта да се адаптира към различни устройства и размери на прозорците на браузъра. Това позволява да се създаде удобен за потребителя и ориентиран към потребителя дизайн за всяка аудитория.

- **Express.js**

Express.js (или Express) е уеб фреймуърк, създадена на JavaScript. С Express може да се създават комплексни динамични уеб приложения, които използват бази данни. Това е най-популярният Node.js фреймуърк за уеб приложения, с основно предназначение за изграждане на уеб приложения и API-ове.



Фигура 8 Express – Лого

Едно от ключовите предимства на Express.js е способността да обслужва огромен брой потребители с лекота, тъй като може да обработва заявки асинхронно в единична нишка. Това го прави особено подходящ за изграждането на приложения, при които се очаква потребителската база да се увеличи бързо и с много.

- **MySQL**

MySQL е най-популярната система за управление на бази данни с отворен код. Тя се разпространява и поддържа от Oracle Corporation. Той използва SQL (Structured Query Language) за достъп и работа с данните, съхранявани в таблици. MySQL е широко използван в уеб приложения и се характеризира със своята скорост, надеждност и гъвкавост.



Фигура 9 MySQL – Лого

MySQL има клиент-сървър мрежова архитектура и по-точно – това е база данни с трислойна архитектура. Машините, на които се инсталира системата, се наричат клиенти. Когато искат достъп до съхраняваната в базата данни информация, трябва да се свържат със сървърите на системата. Клиентът и сървърът са два от трите основни слоя в архитектурата на MySQL. Те се допълват от т.нар. слой за съхранение – storage layer. Той се грижи за съхранението и достъпването на наличните данни. MySQL е с отворен код, което означава, че всеки може да изтегли софтуера от интернет и да го използва безплатно.

Глава III. Проектиране на системата

1. Изисквания на програмната система

Реализацията на уеб платформата се фокусира върху изграждането на система за възпроизвеждане на музикално съдържание, както и различни вариации на търсене и комбиниране. Изглежда, който трябва да представи системата, трябва да е динамичен и тясно свързан с базата данни, която ще доведе до стойностното преживяване на потребителя.

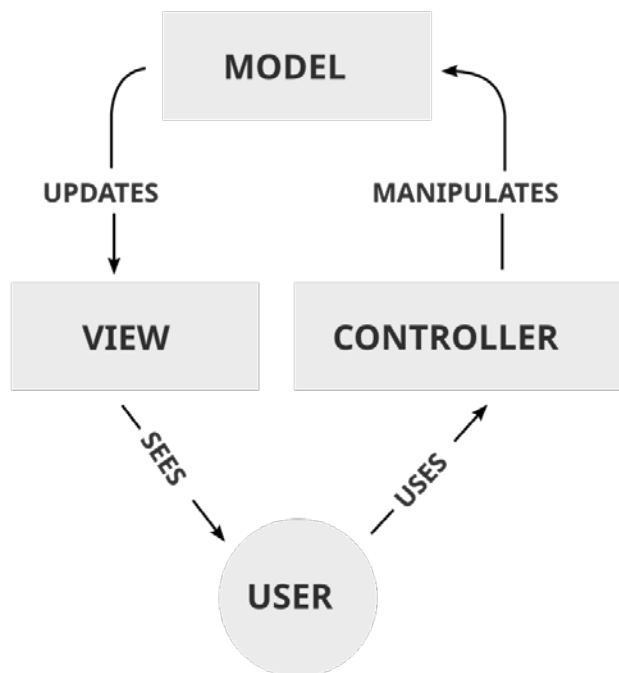
За да се изпълнят тези изисквания трябва се осъществи правилно прилагане на избраните технологии за разработване на системата. Технологиите, които се прилагат по време на разработката са фокусирани върху поддръжката на операциите, които трябва да се извършват, като взаимодействието на потребителите с потребителския интерфейс или заявките за обработка от базите данни, които ще се изпращат, подсигурявайки, че няма да се допускат блокирания.

Организационните правила, които са реализирани в системата изпълняват формата на изисквания, които трябва да се спазят. Те осигуряват съответствие с очакванията на потребителите и организационните цели, гарантирайки стабилна и последователна работа на платформата. Някои от основните бизнес правила в приложението са:

- **Потребителски и администраторски достъп**
 - a. Приложението може да се използва, дори и потребителят да няма създаден акаунт;
 - b. Потребителите могат да влязат в профила си или да се регистрират чрез логин страницата;
 - c. Нормалния администратор може да прави корекции по базата данни, което включва добавяне, премахване и коригиране на изпълнители, албуми и т.н.;
 - d. Главния администратор може да извършва функционалностите на нормалния администратор, както и да създава други администраторски профили.
- **Възпроизвеждане на музикално съдържание**
 - a. Трябва да има музикален плейър, който ще стартира песните;
 - b. Когато се натисне върху песен тя трябва да се стартира в плейъра;
 - c. Трябва да се създава опашка от песни в зависимост от страницата, на която се намира потребителя;
 - d. Трябва да може да се навигира от песен в песен в опашката чрез бутони на плейъра.

- **Търсене и управление на музикално съдържание**
 - a. Потребителите трябва да могат да търсят лесно определени песни;
 - b. Потребителите ще могат да търсят за музика чрез изпълнители, плейлисти и т.н.
- **Управление на плейлисти**
 - a. Потребителите, които са влезли в акаунта си имат право да създадат плейлист, където да добавят песни;
 - b. Плейлистите могат да бъдат както частни, така и публични;
 - c. Само създателя на плейлиста може да променя статуса на плейлиста от публичен на частен и обратно;
 - d. Потребителите могат да преглеждат и последват публични плейлисти;
 - e. Потребителите могат да добавят редактори на плейлиста от своите приятели;
 - f. Потребителя няма да има право на достъп до плейлист, който е частен дори и да го е последвал или да е негов редактор.
- **Приятели**
 - a. Потребителите имат право да изпращат покана за приятелство на други потребители;
 - b. Потребители, които са приятели могат да се добавят като редактори в създадените от тях плейлисти;
 - c. Потребителите могат по всяка време да прекратят приятелството си с друг потребител.
- **Любими песни**
 - a. Потребителите, които са влезли в акаунта си имат право да добавят харесани от тях песни в любими песни;
 - b. Потребителите могат да преглеждат и слушат техните любими песни отделно.
- **Персонализирани препоръки**
 - a. В базата данни трябва да се запазва консумираното музикално съдържание на всеки потребител;
 - b. Трябва да се селектират топ жанровете, които потребителят е слушал в последния месец и да се изкарват песни от тези жанрове;
 - c. Потребителят ще може да преглежда препоръчаните песни и да ги пуска отделно.

2. Архитектура на системата



Фигура 10 Архитектура на системата

„Model-view-controller“ (за по-кратко „MVC“) е архитектурен шаблон, който най-често се използва при създаването на потребителски интерфейс. Той „разделя“ приложението на три взаимосвързани части. Това е направено с цел да се раздели вътрешното представяне на информация от начините, по които информацията се представя на и приема от потребителя. MVC шаблонът разделя тези главни компоненти, което позволява на разработчиците да използват отново вече написан код по-ефективно, а също така позволява и паралелна разработка. За пръв път този шаблон за дизайн е използван в програмния език Smalltalk.

Традиционно MVC шаблонът се използва за графични потребителски интерфейси (*Graphical User Interface – GUI*), но вече той се използва както и за уеб приложения, така и за мобилни такива, а дори и за други типове клиенти.

Model-View-Controller, шаблонът се състои от три отделни компонента, които изпълняват определена роля и са взаимосвързани в определена последователност:

- Модел

Моделът е централен компонент в шаблона, който е независим от потребителския интерфейс. Представява данните, бизнес логиката и правилата на приложението. Той пряко управлява данните, логиката и правилата и отговаря за извличането на данни от база данни, за извършването на операции с тях и за

самоактуализирането им или за уведомяването на изгледа за промени, когато данните се променят.

- Изглед

Потребителският интерфейс на приложението. Представява презентационен слой, който показва данните от модела. Изгледът получава данни от Модела и ги визуализира, за да може потребителят да ги види. Той не съдържа никаква бизнес логика.

- Контролер

Изпълнява ролята на посредник между Модела и Изгледа. Той обработва входните данни на потребителя, обработва ги (като извиква съответните функции в Модела) и определя кой изглед да бъде показан. По същество контролерът актуализира Модела въз основа на потребителския вход и казва на Изгледа какво да се покаже.

Тъй като MVC разделя основните компоненти на приложението, това позволява на разработчиците да работят паралелно по различни компоненти, без да оказват влияние или да си пречат един на друг. Например един разработващ екип може се раздели на две части, като едната поема изграждането на back-end логиката, а другата – на front-end частта. Back-end разработчиците проектират структурата на данните и как потребителят взаимодейства с тях, без да се нуждаят от потребителски интерфейс (front-end), за да тестват и усъвършенстват тези аспекти на приложението. Обратно – front-end разработчиците могат да проектират и тестват изгледа на приложението, преди да имат готова структура от данни, с която той да борава.

Създавайки компоненти, които са независими едни от други, разработчиците могат да използват многократно един компонент в различни приложения. Изгледът на едно приложение може да се преработи бързо и лесно за друго сходно приложение, което борава с коренно различна структура от данни; моделът който обработва данните и който представлява цялостната структура е отделен компонент спрямо изгледа и това дава свобода на разработчиците да използват наново един и същи код, според нуждите си.

Тези негови предимства, които предлага го правят подходящ за сложни и мащабни приложения, като софтуерната система достига ново ниво на сложност, което я прави по-трудно разбираема за софтуерните разработчици, работещи по нея.

3. Логически модел

Чрез логическия модел, проектът може да се представи по систематичен и визуален начин и да се създаде разбиране за взаимоотношенията между ресурсите, които се използват за изпълнението на програмата, функционалностите, които са планирани, и промените или резултатите, които трябва да бъдат постигнати.

Основата на логическия модел се изгражда върху проблема, на който програмата отговаря. След като вече са дефинирани функционалностите, които трябва да бъдат изпълнени и проблемът, който решава системата е ясен, може да се разработи ефективен логически модел.

Основните компоненти, които изграждат логическия модел са:

1. Вход (Ресурси):

- Това включва всички ресурси, необходими за разработка и поддръжка на приложението. Това са инструменти, технологии, екип, и материална база, които се използват за изпълнение и управление на компонентите на системата.

2. Дейности:

- Това са специфичните процеси и задачи, които се извършват за разработката на всеки компонент в MVC. Това включва проектиране, кодиране, тестване и внедряване на модели, интерфейси и контролери.

3. Изход (Резултати):

- Преките продукти от изпълнените дейности. Това са всички компоненти от архитектурата, които са разработени и функционират, като например готови модели, интерфейси и контролери, които могат да бъдат тествани и използвани.

4. Резултати (Краткосрочни, Средносрочни, Дългосрочни):

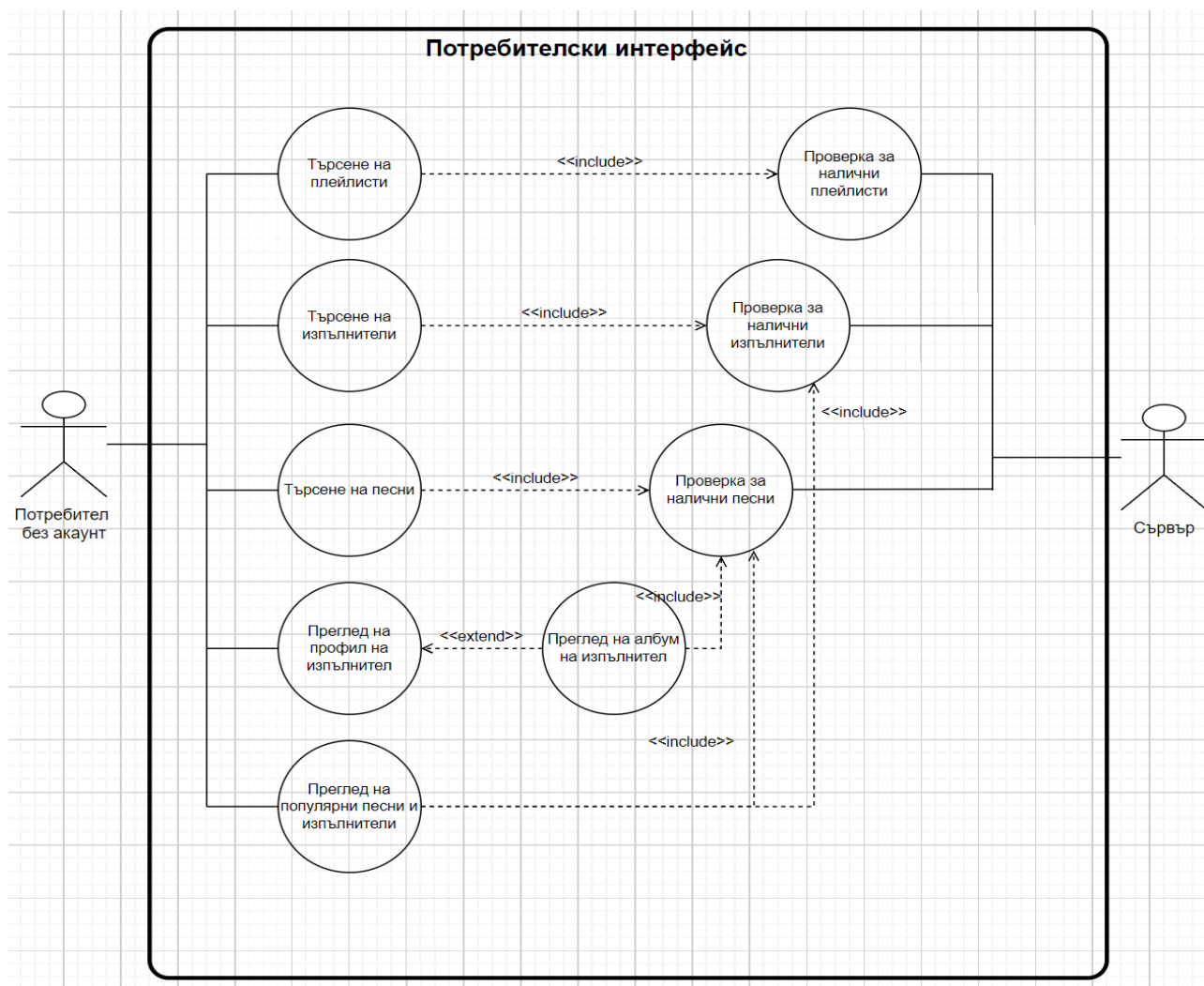
- Това са промените и въздействието, които следват след успешното внедряване на MVC архитектурата. Краткосрочните резултати са свързани с подобрения в разбирането и организацията на кода, средносрочните с ефективност в разработването на нови функции, а дългосрочните с устойчивостта и възможността за разширяване на приложението.

Системата следва да бъде разделена на три главни части – употреба без създаване на профил, потребителска част и администраторска част, като потребителската и администраторската се определят в зависимост от данните, които са въведени в формата за влизане.

Функционалностите, които се присвояват на потребителите, които използват уеб платформата без да си създават потребителски профил са:

- Консумиране на музикално съдържание;
- Търсене на музикално съдържание;
- Откриване на изпълнители;
- Откриване на песни;
- Откриване на публични плейлисти;
- Преглед и слушане на песни от профил на изпълнител;
- Преглед и слушане на песни от албум;
- Преглед и слушане на песни от плейлист с публичен статус.

Чрез UML Use Case диаграма, могат да се визуализират функционалностите на потребителите, които използват системата без да имат профил:

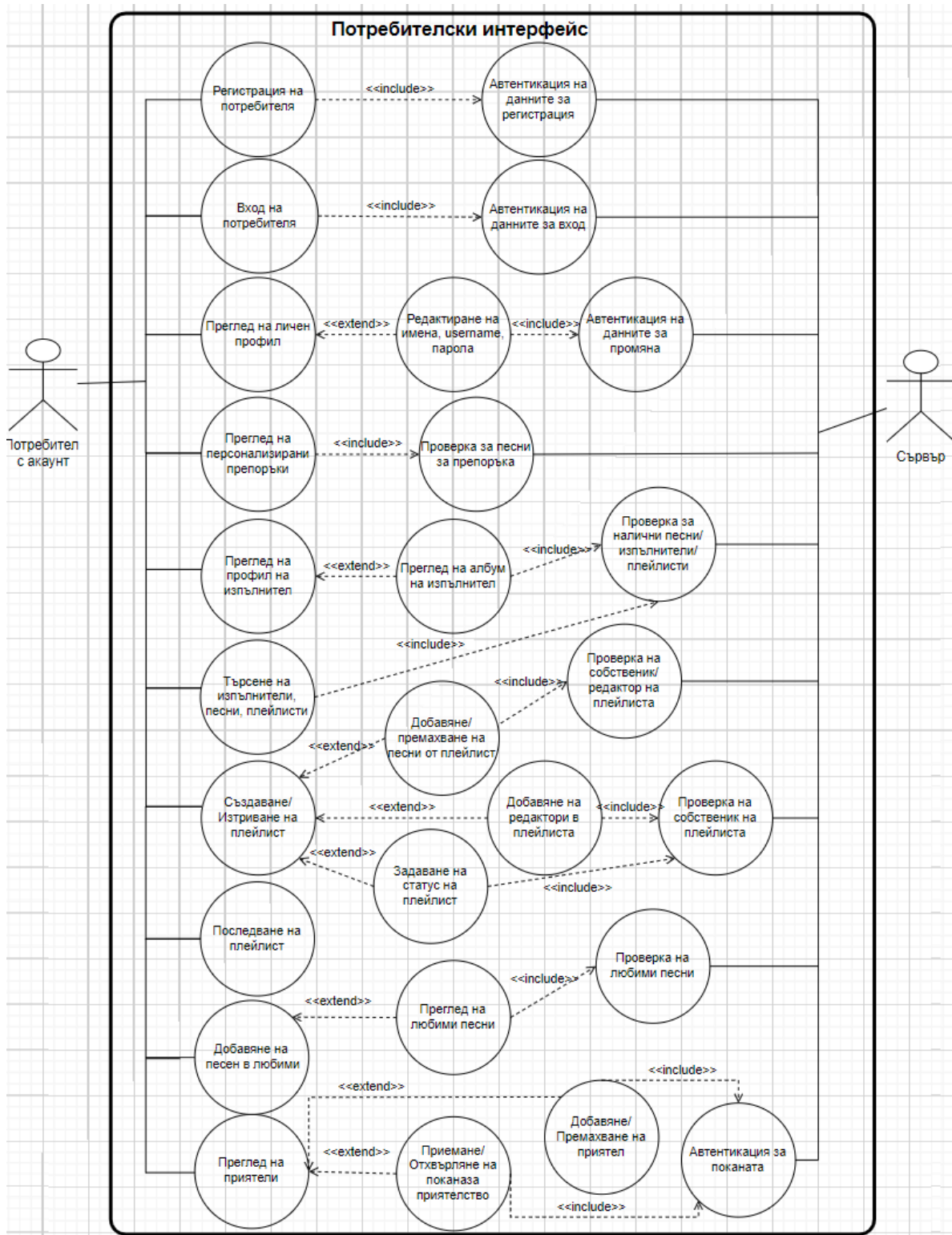


Фигура 11 Use Case диаграма на потребител без профил

Функционалностите, които се присвояват на потребителите, които използват уеб платформата, удостоверили са се и успешно са влезли в профила си са:

- Всички функционалности, които имат потребителите, които ползват платформата без създаден профил;
- Създаване на плейлисти;
- Добавяне и премахване на песни към плейлисти, когато потребителя е създател на плейлиста или е негов редактор;
- Последване на плейлист;
- Сменяне на статуса на плейлист на публичен/частен, само когато е създаден от потребителя;
- Добавяне и премахване на редактори на плейлиста, само когато е създаден от потребителя;
- Преглеждане на профила на потребителя;
- Правене на промени по профила на потребителя, сменяйки имена, username и парола;
- Добавяне на приятели;
- Приемане/Отказване на покани за приятелство;
- Добавяне на песни в любими;
- Отделно слушане на любимите песни на потребителя;
- Преглед на персонализирани предложения, базирани на скорошното консумирано съдържание от потребителя;
- Вход и изход от системата.

Чрез UML Use Case диаграма, могат да се визуализират функционалностите на потребителите, които използват системата, които имат създаден профил:

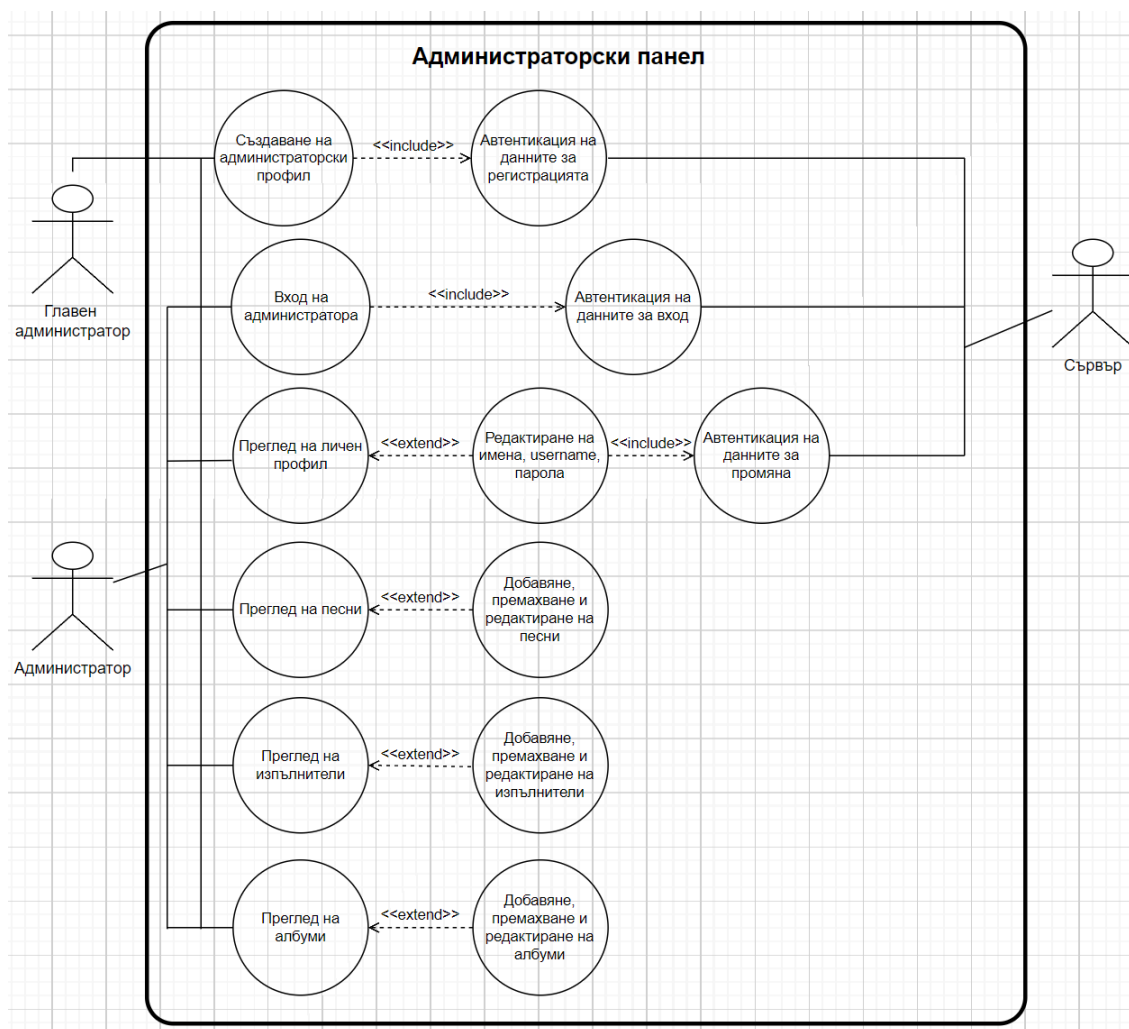


Фигура 12 Use Case диаграма на потребител със създаден профил

Функционалностите, които се присвояват на администраторите са:

- Администраторите имат достъп до всички функционалности на потребителите;
- Добавяне, премахване и преглед на песни в системата;
- Добавяне и премахване и преглед на изпълнители в системата;
- Добавяне и премахване и преглед на албуми в системата;
- Създаване на профил на администратор, достъпно само за главен администратор;
- Вход и изход от системата.

Чрез UML Use Case диаграма, могат да се визуализират функционалностите на администраторите:

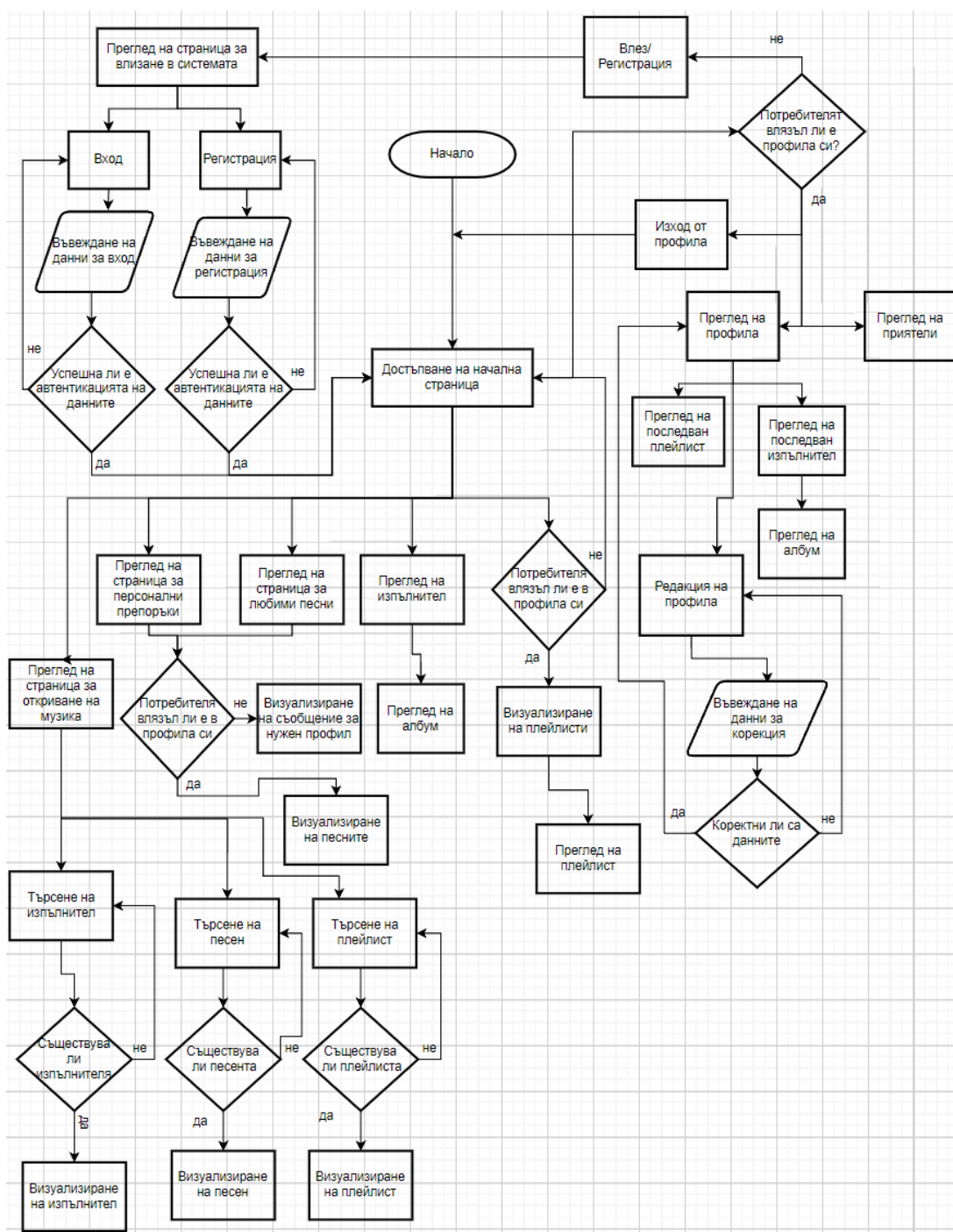


Фигура 13 Use Case диаграма на администратор

4. Диаграма на работния процес

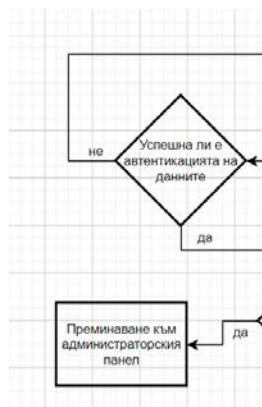
Диаграмата на работния процес е визуално представяне на последователността от стъпки, задачи или процеси, които се изпълняват в дадена система, програма или проект. Тя помага да се разбере как протичат действията, кой ги изпълнява, и какво се случва след всяка стъпка, като се изяснява взаимодействието между действията и участниците в тях.

Flowchart диаграма, представяща работния процес на потребителите:

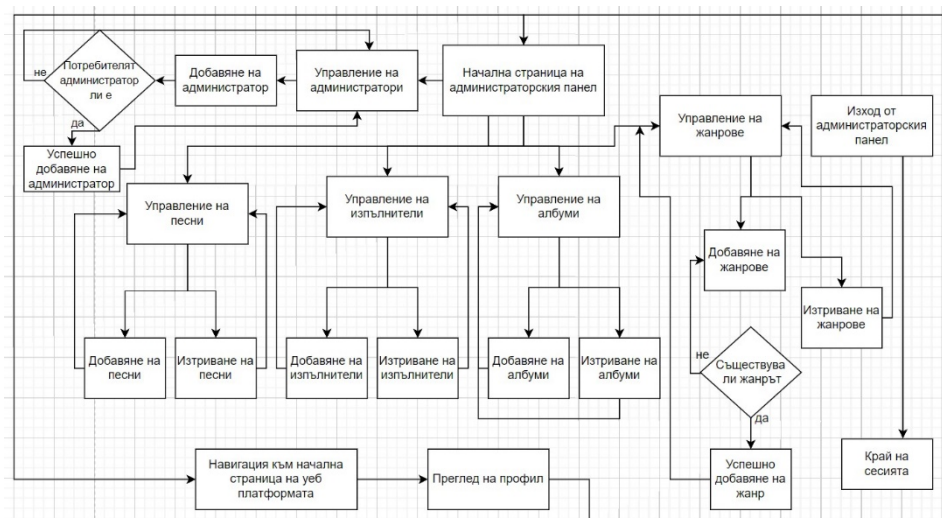


Фигура 14 Flowchart диаграма за потребителя

Flowchart диаграма, представяща работния процес на администраторите:



Фигура 15 Flowchart диаграма за автентикация на администратор



Фигура 16 Flowchart диаграма за администратора - TBD

5. Организация на данните

Разработваната система ще използва сървърна MySQL база данни за управление на информацията. Преди да се пристъпи към изграждането на структурата на тази база, е необходимо ясно да са се формулирали основните цели на системата, както и нейните функции. Стратегическата организация и анализ на данните са от решаващо значение за създаването на стабилна и ефективна архитектура.

Данните в системата ще бъдат представени чрез релационен модел – подход, при който информацията се съхранява под формата на таблици. Тези таблици могат да бъдат свързани помежду си чрез логически зависимости. Всяка таблица съдържа редове (записи), които се състоят от колони (атрибути). За идентификация на записите се използва уникален идентификатор, наречен първичен ключ. Освен него, могат да се използват и външни ключове, които създават връзки между таблиците, но не са задължителни.

Типовете връзки между таблиците могат да бъдат различни:

- Едно към едно: една стойност от една таблица съответства на една стойност в друга;
- Едно към много: един елемент от първата таблица е свързан с множество елементи от втората;
- Много към много: множество записи от двете таблици могат да бъдат свързани помежду си.

Базата данни ще бъде разположена на централен сървър, което позволява на различни клиенти да осъществяват достъп до нея. Обменът на информация се извършва чрез изпращане на заявки до сървъра, който ги обработва, осъществява необходимото взаимодействие с базата и връща отговор за резултата от действието – дали е успешно или не. В нея се съхраняват различни типове данни, нужни за правилното функциониране на системата и нейните основни компоненти.

Концептуалният модел е едно от най-добрите средства за представяне на структурата на една база данни, преди да бъде създадена, без да се навлиза в подробности. Базата данни, която ще бъде създадена и използвана за осъществяване на целите и решаване на задачите на проекта съдържа следната информация:

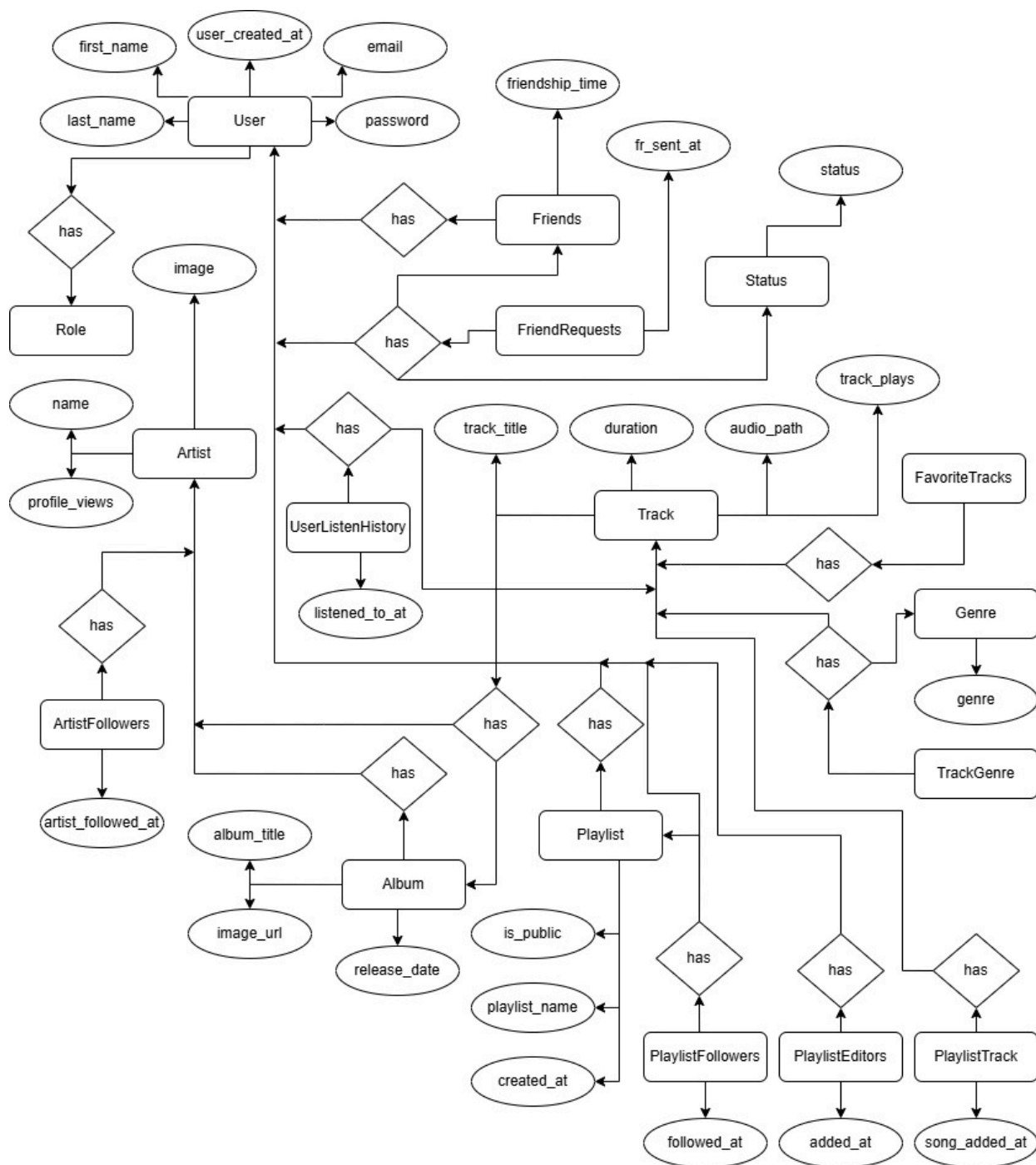
- User
 - user_id - първичен ключ
 - username - потребителско име
 - first_name - име
 - last_name - фамилия
 - password - парола
 - email - имейл
 - user_created_at - дата на създаване на потребителя
 - user_role - роля
- Role
 - role_id - първичен ключ
 - role - роля
- Friends
 - friends_id - първичен ключ
 - friends_user_id - първичен ключ на потребител
 - friends_friend_id - първичен ключ на приятел
 - friendship_time - време на приятелство
- FriendRequests
 - friend_request_id - първичен ключ
 - fr_sender_id - първичен ключ на изпращач
 - fr_receiver_id - първичен ключ на получател
 - status – първичен ключ на статус

- fr_sent_at – време на изпращане на поканата за приятелство
- Artist
 - artist_id - първичен ключ
 - artist_name - име
 - image_url - снимка на изпълнителя
 - profile_views - преглеждания на профила
- ArtistFollowers
 - follower_id - първичен ключ
 - artist_id - първичен ключ на изпълнител
 - user_id - първичен ключ на потребител
 - artist_followed_at - дата на последване на изпълнителя
- Track
 - track_id - първичен ключ
 - track_title - име на песента
 - duration - времетраене
 - audio_path - път на аудио файла
 - album_id - първичен ключ на албум
 - artist_id - първичен ключ на изпълнител
 - track_plays - брой на слушания на песента
- FavoriteTracks
 - user_id - първичен ключ на потребител
 - track_id - първичен ключ на песен
- Genre
 - genre_id - първичен ключ
 - genre - име на жанр
- TrackGenre
 - trackgenre_id - първичен ключ
 - track_id - първичен ключ на песен
 - genre_id - първичен ключ на жанр
- UserListenHistory
 - listen_id - първичен ключ
 - listen_track_id - първичен ключ на песен
 - listen_user_id - първичен ключ на потребител
 - listened_to_at - дата на слушане на песента
- Album
 - album_id - първичен ключ
 - album_title - име на албума
 - release_date - дата на издаване
 - image_url - снимка на албума
 - artist_id - първичен ключ на изпълнител
- Playlist

- playlist_id - първичен ключ
 - playlist_name - име на плейлиста
 - created_by_user_id - първичен ключ на потребител
 - created_at - дата на създаване
 - is_public - статус
- PlaylistFollowers
 - follower_id - първичен ключ
 - playlist_id - първичен ключ на плейлист
 - user_id - първичен ключ на потребител
 - followed_at – дата на последване на плейлиста
- PlaylistEditors
 - pl_editors_id - първичен ключ
 - playlist_id - първичен ключ на плейлист
 - user_id - първичен ключ на потребител
 - added_at - дата на добавяне
- PlaylistTrack
 - playlist_id - първичен ключ на плейлист
 - track_id - първичен ключ на песен
 - song_added_at – дата на добавяне на песента
- Status
 - status_id – първичен ключ на статус
 - status - статус

Включването на база данни в системата служи за надеждно управление на информацията, като дава възможност на потребителите да я преглеждат и използват при нужда. Чрез базата се гарантира по-високо ниво на сигурност, тъй като достъпът до данните може да бъде контролиран и защитен срещу неразрешено използване.

След като информацията за концептуалният модел на базата данни е описана, може да се създаде Entity-Relationship диаграма на този модел:



Фигура 17 Entity-Relationship диаграма на концептуален модел на базата данни

Диаграмата на Чен и нейните нотации се фокусират върху ясно идентифициране на обектите, техните атрибути и връзките между тях, което тази Entity-Relationship диаграма ефективно илюстрира. Обектите са представени чрез правоъгълници. Атрибутите са с кръгла форма и правят връзка към обектите. Връзките между обектите се обозначават с форма на ромб и текст.

Тъй като избрания модел за системата е релационен, то това предоставя много предимства като - опростеност, цялостност на данните, сигурност и гъвкавост. Главните функции, които ще се използват и реализират в системата са така нар. CRUD операции. CRUD операциите се използват в приложения за постоянно съхранение, което означава, че тези приложения ще запазят данните си дори след като системата се изключи. Те са различни от операциите с данни, съхранявани в енергозависими хранилища, като например Random Access Memory или кеш файлове. CRUD операциите се разделят на заявки за – създаване, четене, актуализиране, изтриване:

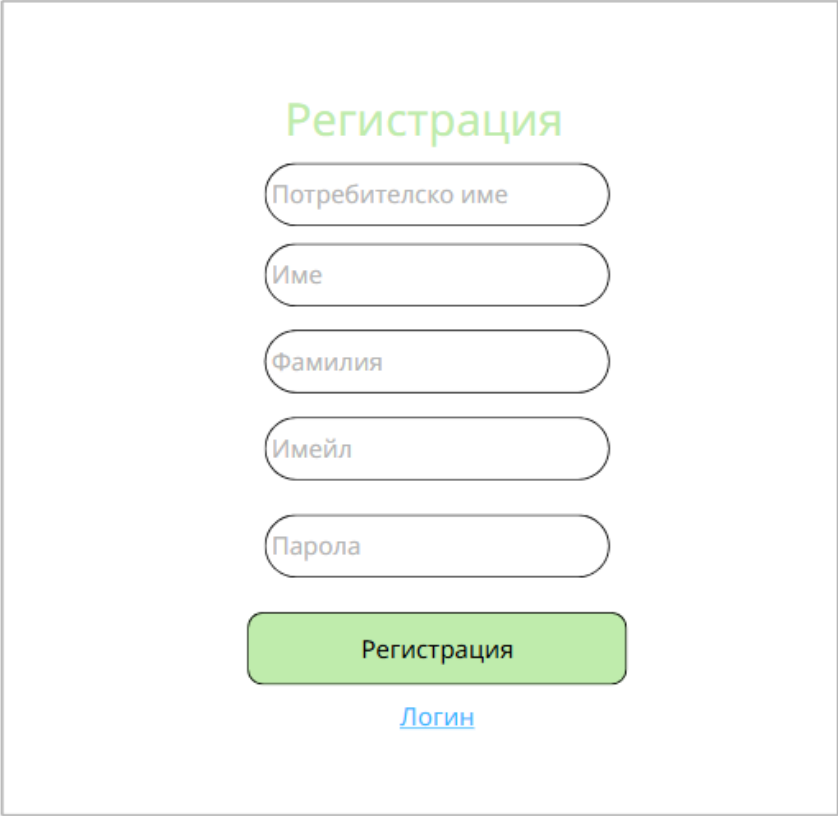
- **Операция създаване – Create** - Функцията за създаване позволява на потребителите да добавят нов запис в базата данни. В приложението за релационни бази данни SQL, функцията за създаване се нарича INSERT. В Oracle HCM Cloud тя се нарича create. Записът представлява ред, а колоните се наричат атрибути. Потребител може да създаде нов ред и да го попълни с данни, съответстващи на всеки атрибут, но само администратор би могъл да добави нови атрибути към самата таблица.
- **Операция четене – Read** - Функцията за четене е подобна на търсеща функция. Тя позволява на потребителите да търсят и извличат специфични записи в таблицата и да прочитат техните стойности. Потребителите могат да намерят желаните записи, използвайки ключови думи или чрез филтриране на данните на базата на персонализирани критерии.
- **Операция актуализиране – Update** - Функцията за актуализиране се използва за модифициране на съществуващи записи в базата данни. Потребителите може да трябва да променят информация в няколко полета, за да изменят запис. В SQL и Oracle HCM Cloud функцията за актуализиране се нарича „Update“.
- **Операция изтриване – Delete** - Функцията за изтриване позволява на потребителите да премахват записи от база данни, които вече не са нужни. SQL и Oracle HCM Cloud имат функция за изтриване, която позволява на потребителите да изтриват един или повече записи от базата данни. Някои приложения за релационни бази данни могат да позволят на потребителите да извършат твърдо или меко изтриване. Твърдото изтриване премахва записите от базата данни завинаги, докато мекото изтриване може да актуализира статуса на ред, за да покаже, че е бил изтрит, оставяйки данните на място и непокътнати.

6. Макет на платформата

След приключване на аналитичната фаза на проекта и дефиниране на основните функционални изисквания, следващата стъпка е създаването на визуален макет на платформата. Този макет представлява предварителен прототип, който цели да илюстрира структурата, дизайна и основната логика на потребителския интерфейс. Чрез него се визуализират ключовите елементи на платформата, като се осигурява ясна представа за взаимодействието между потребителя и системата още преди реалната ѝ разработка.

Макетът служи като важен инструмент за валидация на концепцията, подпомага идентифицирането на потенциални проблеми в ранна фаза и улеснява комуникацията между екипите. Той не включва пълната функционалност на системата, но представя нейната визия и логика по достъпен и визуално разбираем начин.

Реализирани прототипи в Canva:



Регистрация

Потребителско име

Име

Фамилия

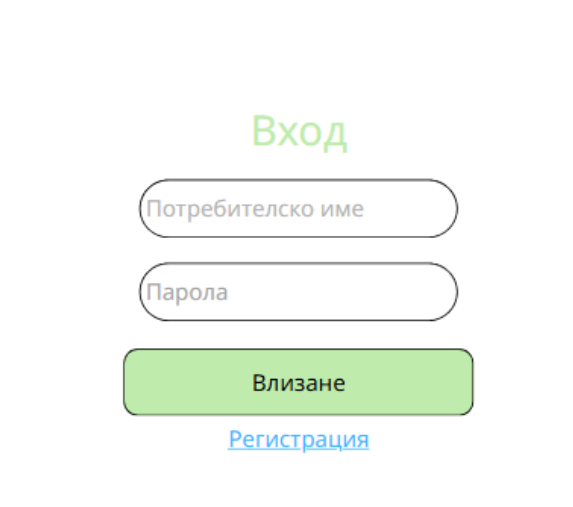
Имейл

Парола

Регистрация

[Логин](#)

Фигура 18 Прототип на страница за регистрация



Вход

Потребителско име

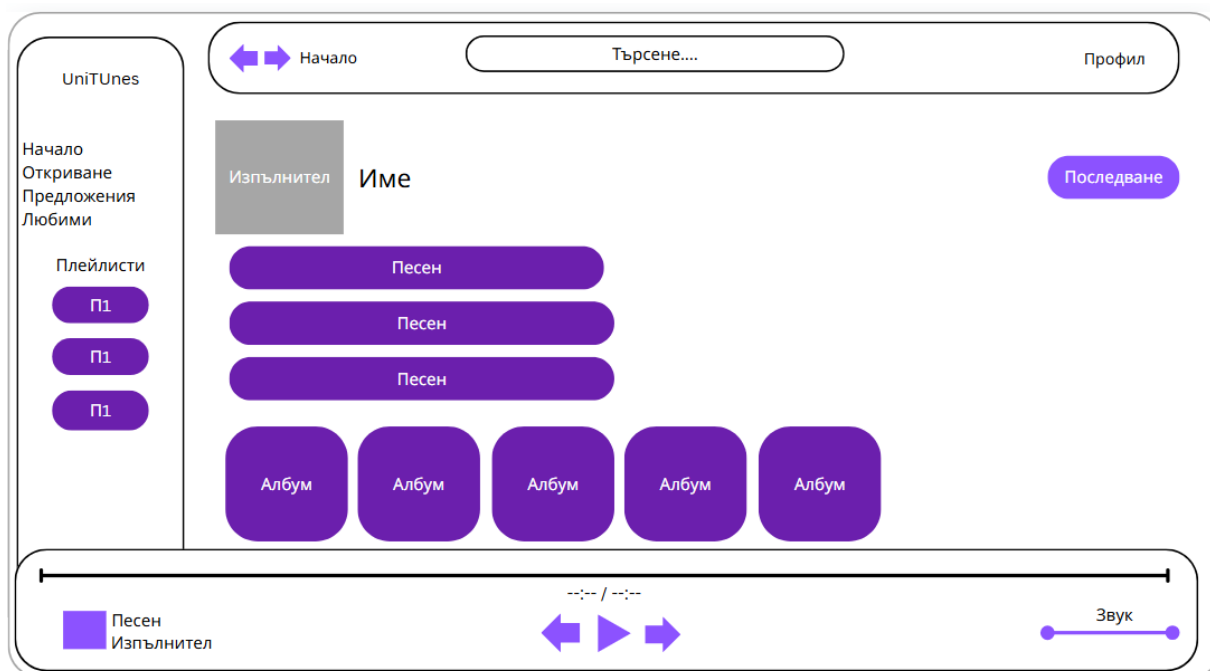
Парола

Влизане

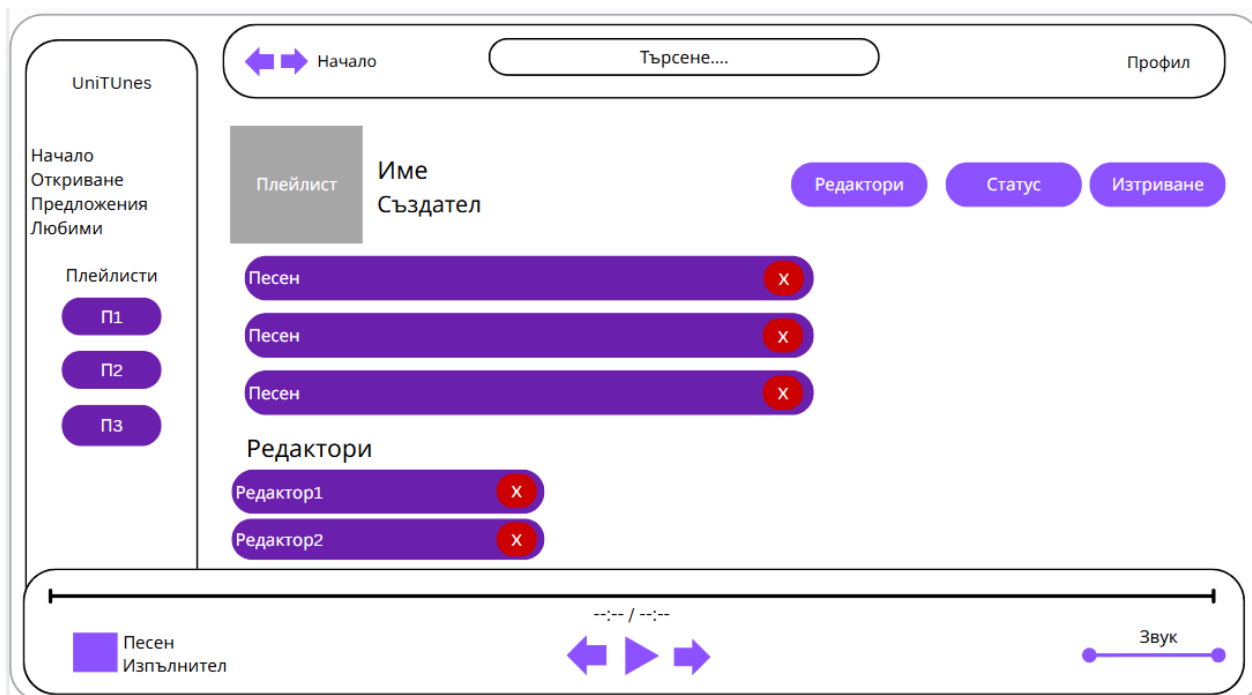
[Регистрация](#)

The image shows a music application interface with the following components:

- Top Bar:** Contains a search bar labeled "Търсене....", a "Начало" (Home) button with a double arrow icon, and a "Профил" (Profile) button.
- Left Sidebar:**
 - Header: "iTunes"
 - Categories: "Начало", "Откриване", "Предложения", "Любими"
 - Section: "Плейлисти" (Playlists) with three items, each labeled "П1" inside a rounded rectangle.
- Main Content Area:** A 3x5 grid of 15 purple rounded rectangles, each labeled "Песен" (Song).
- Bottom Bar:**
 - Progress bar: A horizontal line with a vertical marker.
 - Buttons: Play/pause (two arrows) and Next (right arrow).
 - Volume control: A slider labeled "Звук" (Sound).
 - Legend: A purple square labeled "Песен" and "Изпълнител" (Artist).



Фигура 21 Прототип на страница за изпълнител



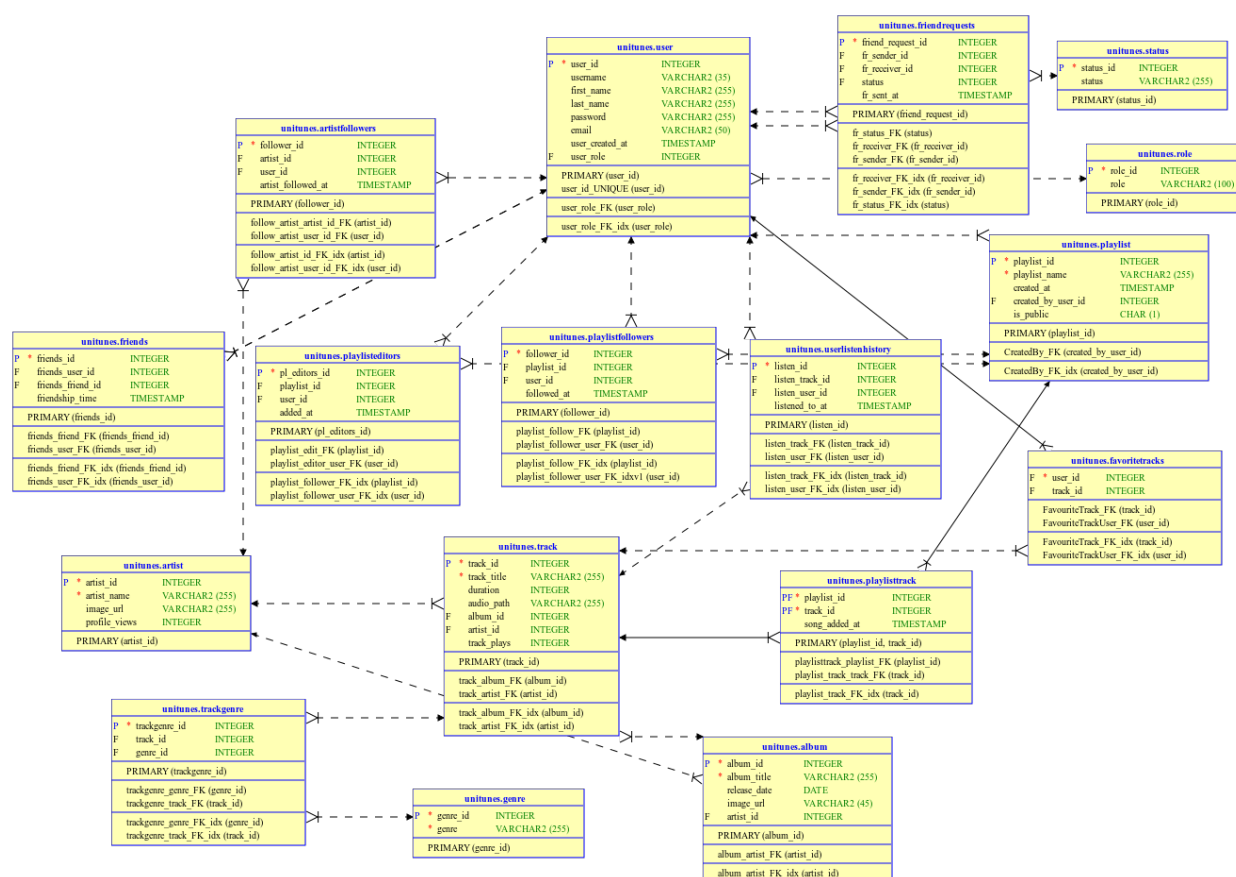
Фигура 22 Прототип на страница за плейлист

Глава IV - Реализация на системата

1. Структура на данните

Системата е разработена, чрез реализиране на заявки през MySQL Workbench. MySQL Workbench предлага няколко предимства за управление и развитие на бази данни. Тя осигурява визуален интерфейс за проектиране, моделиране и управление на бази данни, като по този начин опростява сложните задачи за администраторите на и разработчиците. Основните предимства включват визуален дизайн на схемите, възможности за direct и reverse инженерство и ефективно управление на промените. Тя също така предоставя инструменти за анализ и оптимизация на производителността.

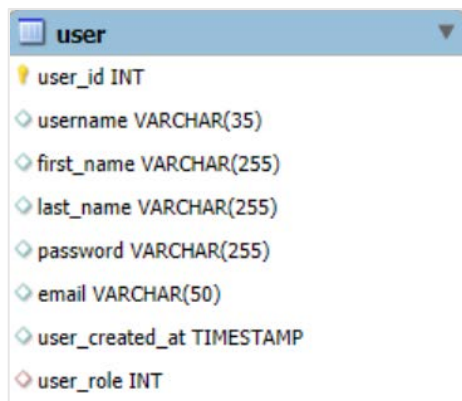
1.1. Нормализация на таблиците от БД



Фигура 23 Релационен модел на базата данни

1.2. Физически модел и описание на таблиците от БД

Базата данни се състои от следните таблици:



user	
user_id	INT
username	VARCHAR(35)
first_name	VARCHAR(255)
last_name	VARCHAR(255)
password	VARCHAR(255)
email	VARCHAR(50)
user_created_at	TIMESTAMP
user_role	INT

Фигура 24 Таблица User

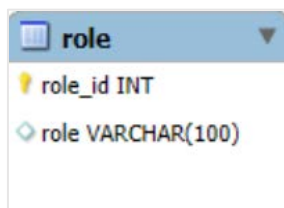
- Таблица Users

Таблицата представлява цялата информация за всеки регистриран потребител.

Поleta в таблицата:

- user_id – INT – първичен ключ
- username – VARCHAR(35) – Поле с текст, което се използва за потребителското име на даден потребител или администратор, ограничено до 35 символа
- first_name – VARCHAR(255) – поле с текст за името на потребител
- last_name – VARCHAR(255) – поле с текст за фамилията на потребител
- password – VARCHAR(255) – поле с текст за паролата, зададена от потребител
- email – VARCHAR(50) – поле с текст за имейла на даден потребител, зададен при създаването на акаунта
- user_created_at – TIMESTAMP – поле за дата и час, съхранява датата и часа на създаване на потребителя. При извличане стойността се връща във формат 'yyyy-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа
- user_role – INT – външен ключ (foreign key), който сочи към таблицата Role, с който се извлича информация за ролята на потребителя

- Таблица Role

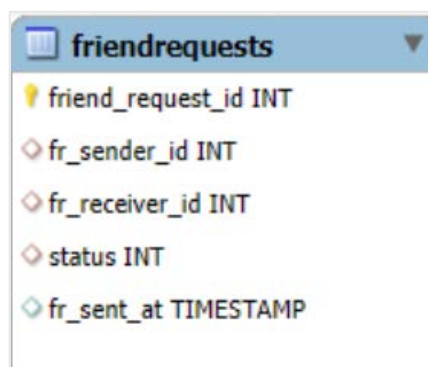


Фигура 25 Таблица Role

Тази таблица съдържа информация за всички роли, които могат да притежават потребителите. При създаване са въведени роли: “user”, “admin_normal”, “admin_advanced”

Полета в таблицата:

- role_id – INT - първичен ключ
- role – VARCHAR(100) – поле с текст, което съдържа информация за конкретна роля, която потребител може да притежава



Фигура 26 Таблица FriendRequests

- Таблица FriendRequests

Тази таблица съдържа информация за изпратени покани за приятелство от един потребител до друг.

Полета в таблицата:

- friend_request_id – INT - първичен ключ

- fr_sender_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителя~~т~~, който изпраща поканата за приятелство
- fr_receiver_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителя~~т~~, който получава поканата за приятелство
- status – INT – външен ключ (foreign key), който сочи към таблицата Status, за да се извлече статус~~ът~~ на изпратената покана
- fr_sent_at – TIMESTAMP – поле за дата и час, съхранява информация за кога е ~~изпратена~~изпратена поканата за приятелство. При извличане стойността се връща във формат 'уууу-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа



Фигура 27 Таблица Friends

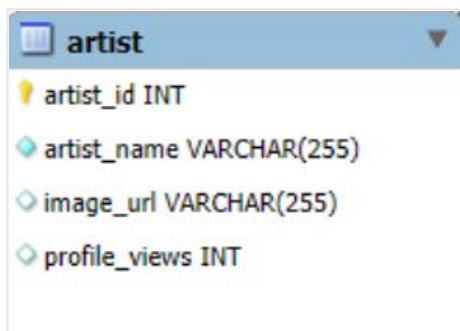
- Таблица Friends

Тази таблица съдържа информация за двама потребители, които са приятели, тоест статуса на поканата за приятелство изпратена от единия към втория е приета.

Поleta в таблицата:

- friends_id – INT - първичен ключ
- friends_user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече първия потребител от приятелите
- friends_friend_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече втория потребител от приятелите
- friendship_time – TIMESTAMP – поле за дата и час, използва се за задаване на дата и час на началото на приятелството на двама потребители. При извличане стойността се връща във формат 'уууу-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент

(CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа



Фигура 28 Таблица Artist

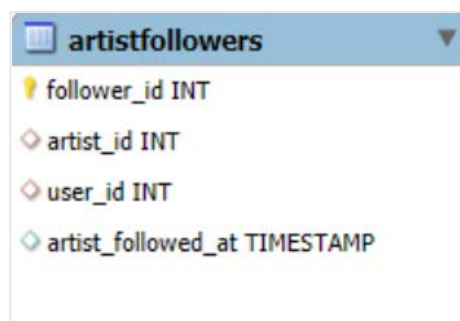
- Таблица Artist

Тази таблица съдържа информация за всички изпълнители.

Поleta в таблицата:

- artist_id – INT - първичен ключ
- artist_name – VARCHAR(255) – поле с текст, съхраняващо името на изпълнител
- image_url – VARCHAR(255) – поле с текст, съхраняващо пътя до снимката на изпълнител
- profile_views – INT – числово поле, в което се съхранява броя посещения на профила на даден изпълнител

- Таблица ArtistFollowers



artistfollowers	
⚡	follower_id INT
◊	artist_id INT
◊	user_id INT
⌚	artist_followed_at TIMESTAMP

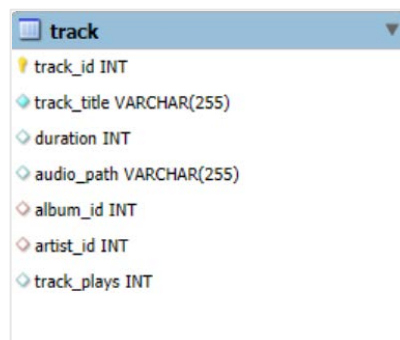
Фигура 29 Таблица ArtistFollowers

Тази таблица съдържа информация за изпълнителите, които потребител следва.

Полета в таблицата:

- follower_id – INT - първичен ключ
- artist_id – INT - външен ключ (foreign key), който сочи към таблицата Artist, за да се извлече изпълнителят, който ще бъде последван
- user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителят, който следва конкретния изпълнител
- artist_followed_at – TIMESTAMP – поле за дата и час, което ~~съхранява~~ съхранява – съхранява кога е последван даден изпълнител. При извличане стойността се връща във формат 'yyyy-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа.

- Таблица Track



track	
track_id	INT
track_title	VARCHAR(255)
duration	INT
audio_path	VARCHAR(255)
album_id	INT
artist_id	INT
track_plays	INT

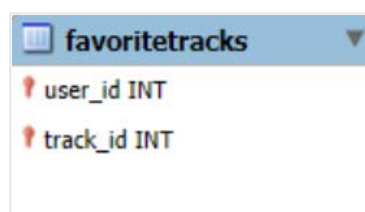
Фигура 30 Таблица Track

Тази таблица съдържа цялата информация за песните.

Поleta в таблицата:

- track_id – INT - първичен ключ
- track_title – VARCHAR(255) – поле с текст, което съдържа името на песен
- duration – INT – числово поле, което съдържа продължителността на песен
- audio_path – VARCHAR(255) – поле с текст, съхраняващо пътя до аудио файл на песен
- album_id – INT - външен ключ (foreign key), който сочи към таблицата Album, за да се извлече албумът от който е дадена песен
- artist_id – INT - външен ключ (foreign key), който сочи към таблицата Artist, за да се извлече изпълнителят, автор на песента.
- track_plays – INT – числово поле, съдържащо броя на слушания на песента

- Таблица FavoriteTracks



favoritettracks	
user_id	INT
track_id	INT

Фигура 31 Таблица FavoriteTracks

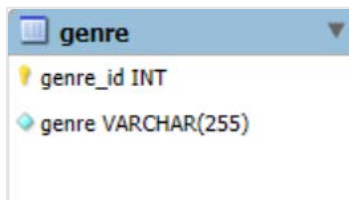
Тази таблица съдържа информация за любимите песни на конкретен потребител.

Поleta в таблицата:

- user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителят, който е добавил песента в любими

- track_id – INT - външен ключ (foreign key), който сочи към таблицата Track, за да се извлече песента, която е добавена в любими

- Таблица Genre

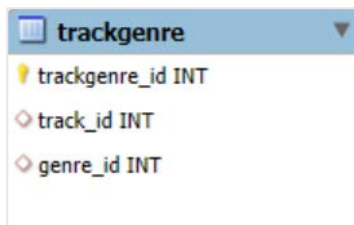


Фигура 32 Таблица Genre

Тази таблица съдържа информация за видовете жанрове, като някои от зададените при създаване са: рок, поп, фънк, метъл, инди, диско, експериментална.

Поleta в таблицата:

- genre_id – INT - първичен ключ
- genre – VARCHAR(255) – поле с текст, ~~съхранява~~ съхранява ~~не съхранява~~ името на конкретен жанр, който една песен може да притежава



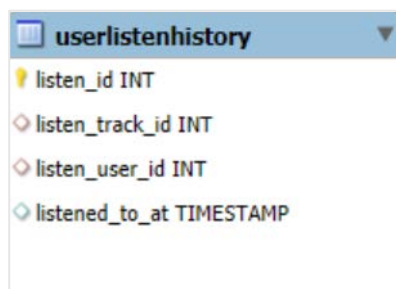
Фигура 33 Таблица TrackGenre

- Таблица TrackGenre

Тази таблица съдържа информация за всичките жанрове на някоя конкретна песен.

Поleta в таблицата:

- trackgenre_id – INT - първичен ключ
- track_id – INT - външен ключ (foreign key), който сочи към таблицата Track, за да се извлече конкретната песен
- genre_id – INT - външен ключ (foreign key), който сочи към таблицата Genre, за да се извлече конкретния ж жанр, който ще се зададе на песента



Фигура 34 Таблица UserListenHistory

- Таблица UserListenHistory

Тази таблица съдържа информация за слушаните песни на конкретен потребител.

Поleta в таблицата:

- listen_id – INT - първичен ключ
- listen_track_id – INT - външен ключ (foreign key), който сочи към таблицата Track, за да се извлече песента, която ще бъде добавена към слушани
- listen_user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителят, слушал песента
- listened_to_at – TIMESTAMP - поле за дата и час, което ~~съхранява~~съхранява ~~съхранява~~ съхранява кога е слушана дадена песен. При извличане стойността се връща във формат 'yyyy-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа

- Таблица Album



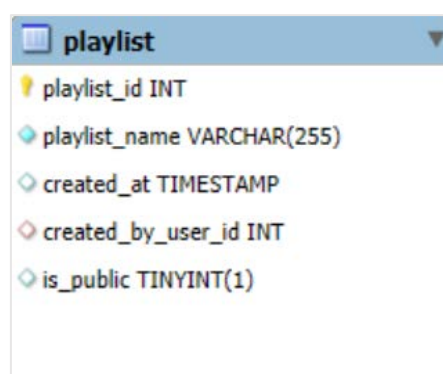
album	
album_id	INT
album_title	VARCHAR(255)
release_date	DATE
image_url	VARCHAR(255)
artist_id	INT

Фигура 35 Таблица Album

Тази таблица съдържа цялата информация за албумите.

Поleta в таблицата:

- album_id – INT - първичен ключ
- album_title – VARCHAR(255) – поле с текст, съхранявасъхранява не съхраняващо името на албум
- release_date – DATE – поле с дата, което съхранява датата, на която е издаден албум
- image_url – VARCHAR(255) – поле с текст, което съхранява пътя до снимката на албум
- artist_id – INT - външен ключ (foreign key), който сочи към таблицата Artist, за да се извлече изпълнителят издал албума



playlist	
playlist_id	INT
playlist_name	VARCHAR(255)
created_at	TIMESTAMP
created_by_user_id	INT
is_public	TINYINT(1)

Фигура 36 Таблица Playlist

- Таблица Playlist

Тази таблица съдържа информация за създадените плейлисти.

Полета в таблицата:

- playlist_id – INT - първичен ключ
- playlist_name – VARCHAR(255) – поле с текст, съхраняващо името на плейлист
- created_at – TIMESTAMP - поле за дата и час, което ~~съхранява~~съхранява кога е създаден даден плейлист. При извличане стойността се връща във формат 'yyyy-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа
- created_by_user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителят, създал плейлиста
- is_public - TINYINT(1) – числово поле, което ~~съхранява~~съхранява дали плейлистът е публичен или частен. Приема стойности 0 или 1, като стойността по подразбиране е 0 (частен)

- Таблица PlaylistFollowers



playlistfollowers	
🔑	follower_id INT
🔗	playlist_id INT
🔗	user_id INT
🕒	followed_at TIMESTAMP

Фигура 37 Таблица PlaylistFollowers

Тази таблица съдържа информация за потребителите, които следват даден плейлист.

Полета в таблицата:

- follower_id – INT - първичен ключ
- playlist_id – INT - външен ключ (foreign key), който сочи към таблицата Playlist, за да се извлече плейлистът, който е последван
- user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да се извлече потребителя, който е последвал плейлиста
- followed_at – TIMESTAMP - поле за дата и час, което ~~съхранява~~съхранява кога е последван даден плейлист. При извличане стойността се връща във формат 'yyyy-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа

- Таблица PlaylistEditors



playlisteditors	
pl_editors_id	INT
playlist_id	INT
user_id	INT
added_at	TIMESTAMP

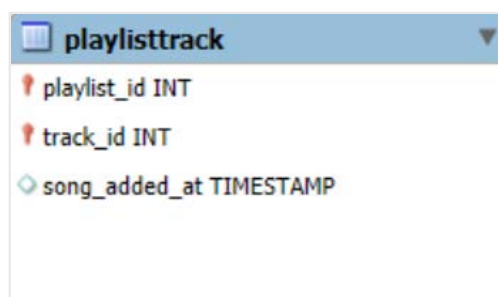
Фигура 38 Таблица PlaylistEditors

Тази таблица съдържа информация за редакторите на конкретен плейлист.

Полета в таблицата:

- pl_editors_id – INT - първичен ключ
- playlist_id – INT - външен ключ (foreign key), който сочи към таблицата Playlist, за да извлече плейлист, на който потребител е редактор
- user_id – INT - външен ключ (foreign key), който сочи към таблицата User, за да извлече потребител, който може да редактира даден плейлист
- added_at – TIMESTAMP - поле за дата и час, което ~~съхранява~~съхранява кога е добавен даден потребител като редактор на плейлист. При извличане стойността се връща във формат 'уууу-мм-дд hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа

- Таблица PlaylistTracks



playlisttrack	
playlist_id	INT
track_id	INT
song_added_at	TIMESTAMP

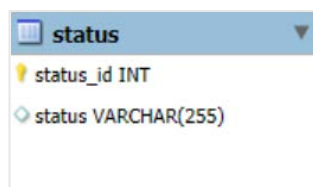
Фигура 39 Таблица PlaylistTrack

Тази таблица съдържа информация за песните в даден плейлист.

Полета в таблицата:

- playlist_id – INT - външен ключ (foreign key), който сочи към таблицата Playlist, за да се извлече плейлиста, където е добавена песента
- track_id – INT - външен ключ (foreign key), който сочи към таблицата Track, за да се извлече песента, която е добавена
- song_added_at – TIMESTAMP- поле за дата и час, което съхранява кога е добавена дадена песен в плейлист. При извличане стойността се връща във формат 'yyyy-mm-dd hh:mm:ss'. Стойността по подразбиране е зададена на текущия момент (CURRENT_TIMESTAMP) и се попълва автоматично при създаване на записа

- Таблица Status



status	
status_id	INT
status	VARCHAR(255)

Фигура 40 Таблица Status

Тази таблица съдържа информация за статуса на конкретна покана за приятелство.

Поleta в таблицата:

- status_id – INT – първичен ключ
- status – VARCHAR(255) – поле с текст, което съдържа конкретния статус на дадена покана за приятелство

2. Описание на програмните модули

Тъй като , приложението е организирано според принципите на MVC (Model-View-Controller), то включва следните програмни модули и директории:

- **Config** - Конфигурации - Съдържа конфигурационни файлове, необходими за стартиране и работа на приложението.
 - dbConfig.js – файлът се използва за инициализация на връзката с базата данни, като стойностите за параметрите на връзката се взимат от локалния файл с конфигурационните променливи
- **Controllers** - Контролери - Контролерите обработват входящите заявки от клиентите, извикват съответните модели, след което връщат резултат, често чрез рендиране на изглед или изпращане на JSON.

- albumController.js
- artistController.js
- authController.js
- favoritesController.js
- friendsController.js
- musicController.js
- playlistController.js
- adminController.js
- **Middleware** - Междинен интеграционен софтуер/Мидълуер - Слой между заявката и нейната крайна обработка – използва се за предварителна валидация, удостоверяване, качване на файлове и др.
 - authenticate.js – файлът съдържа функция, която проверява дали потребителят е влязъл в системата. Той търси специален токен (JWT), който се съхранява чрез бисквитката. Ако токенът е валиден, извлича информацията за потребителя и я запазва в заявката (req.user), за да може другите части на приложението да я използват. Ако токенът липсва или е невалиден, достъпът се отказва с грешка 401, тоест неоторизиран достъп.
 - optionalAuth.js – този файл съдържа функция, която проверява дали потребителят има валиден токен, но не е задължително да има такъв. Ако токен съществува и е валиден се добавя потребителската информация в req.user. Ако няма токен или той е невалиден просто се продължава нататък, без да прекратява заявката.
- **Models** - Модели - Моделите представляват слоя, който отговаря за обработката и достъпа до данните в приложението. Те съдържат функциите, които изпълняват заявки към базата данни, извличат, създават, обновяват или изтриват информация.
 - Album.js –
 - fetchAlbumById – функция, която намира и връща основната информация за албум по зададено id – заглавие, дата на издаване, обложка и ID на изпълнителя
 - fetchAlbumTracks – функция, която извлича всички песни от даден албум, подредени по най-слушани. Включва информация за всяка песен, път до аудиофайла и обложката на албума
 - Artist.js –
 - fetchFollowedArtists – тази функция извлича списък с изпълнителите, които потребителят следва
 - fetchAllArtists – тази функция връща всички изпълнители от базата с цялостната им информация

- `incrementViews` – тази функция увеличава броя на преглежданията на профила на даден изпълнител
 - `fetchArtistById` – тази функция намира конкретен изпълнител по `id` и връща информация за него
 - `fetchArtistTracks` – тази функция извлича най-слушаните песни на даден изпълнител, заедно с данни за албумите, към които принадлежат
 - `fetchArtistAlbums` – тази функция връща албумите на изпълнителя, подредени по дата на издаване
 - `addFollow` – тази функция добавя изпълнител в списъка на последвани от потребителя
 - `removeFollow` – тази функция премахва следването на изпълнител от даден потребител
 - `checkFollowing` – тази функция проверява дали даден потребител вече следва определен изпълнител
- `Favorites.js` –
- `fetchAllFavoriteTracks` – функция, която извлича всички песни, които потребителят е добавил като любими, заедно с цялата информация за изпълнителя на песента и албумът от която е.
 - `getUserFavoritesIds` – функция, която връща само `id`-тата на любимите песни на потребителя
 - `addTrackToFavorites` – функция, която добавя песен към списъка с любими за даден потребител
 - `removeTrackFromFavorites` – функция, която премахва песен от любимите на потребителя
- `Friends.js` –
- `createFriendRequest` – тази функция изпраща покана за приятелство от един потребител към друг със зададен статус при изпращане – „в очакване“
 - `findUserByUsername` – тази функция търси потребител по потребителско име и връща неговото `id`
 - `checkAlreadyFriends` – тази функция проверява дали двама потребители вече са приятели
 - `checkExistingRequest` – тази функция проверява дали вече има изпратена покана между двама потребители, която обаче не е още приета
 - `fetchFriendRequests` – тази функция извлича всички текущи покани за приятелство, които потребителят е получил
 - `fetchFriends` – тази функция връща списък с всички приятели на даден потребител

- acceptRequest – тази функция приема покана за приятелство от даден потребител, обновява нейното състояние и създава двупосочна връзка в таблицата за приятели
 - rejectRequest – тази функция отказва покана за приятелство, като променя състоянието и на „отказана“
 - removeFriendship – тази функция изтрива съществуващото приятелство на двама потребители
- Music.js –
- getTracks – функция, която връща информация за песни, ако е подадено търсене, извежда съвпадения по заглавие на песента
 - getTrendingTracks – функция, която връща топ 20 песни, подредени по най-много слушания
 - getAllTrendingTracks – функция, която извежда до 360 песни с най-много слушания
 - getArtists – функция, която връща до 30 артиста с най-много гледания на профила
 - incrementPlays – функция, която увеличава броя слушания на дадена песен с едно, при нейно пускане
 - recordListenedTrack – функция, която записва, че даден потребител е слушал дадена песен, което ще послужи за изграждане на история от песни, статистики и бъдещи препоръки
 - getRecentListenedToTracks – функция, която извлича най-слушаните жанрове от потребителя през последните 30 дни и предлага до 50 нови песни от същите жанрове, избрани на случаен принцип
- Playlist.js –
- addNewPlaylist – тази функция създава нов плейлист, съхранявайки името, потребителя и времето на създаване.
 - addedTrackToPlaylist – тази функция добавя песен в плейлист само ако потребителят е собственик или редактор
 - fetchedPlaylists – тази функция връща всички плейлисти, до които потребителят има достъп - собствени, редактирани или последвани
 - fetchTracksByPlaylistId – тази функция връща песните в даден плейлист, включително информация за албум и изпълнител.
 - fetchedPublicPlaylists – тази функция извлича всички публични плейлисти, създадени от който и да е потребител

- `fetchOwnedAndEditablePlaylists` – тази функция показва плейлисти, които потребителят е създал или има право да редактира
 - `fetchFollowedPlaylists` – тази функция връща плейлистите, които потребителят следва
 - `addFollowToPlaylist` – тази функция добавя потребителя като последовател на даден плейлист
 - `removeFollowFromPlaylist` – тази функция премахва следването на плейлист от потребителя
 - `deletePlaylist` – тази функция изтрива плейлист и свързаните му записи - песни, последователи, редактори, ако потребителят е собственик
 - `addPlaylistEditor` – тази функция дава правото на друг потребител да редактира плейлиста. Само създателят може да добавя редактори към своите плейлисти
 - `fetchPlaylistEditors` – тази функция извлича всички редактори на даден плейлист
 - `removePlaylistEditor` – тази функция премахва редактор от плейлиста, само ако потребителят е собственик
 - `fetchPlaylistById` – тази функция връща информация за конкретен плейлист по id на плейлиста
 - `updatePlaylistPrivacy` – тази функция променя статуса на даден плейлист от частен на публичен и обратно, като това действие е разрешено само за създателя на плейлиста
 - `fetchUserFriendsList` – тази функция връща списък с приятелите на потребителя, като се използва за списък с възможни редактори за добавяне от създателя на конкретния плейлист
 - `checkIfUserIsFollowingPlaylist` – тази функция проверява дали потребителят следва даден плейлист
 - `removeTrackFromPlaylist` – тази функция премахва песен от плейлист, ако потребителят е създател или редактор
- `User.js` –
- `registerNewUser` – функция, която създава нов потребител, като проверява за съществуващ потребител с това име или имейл. Паролата се хешира чрез `bcrypt`, което представлява функция за хеширане на пароли
 - `getUserByUsername` – функция, която връща потребител по потребителско име, използва се при логин, а ако няма такъв се изписва съобщение за грешка

- `updatedUserProfile` – функция, която позволява на потребителя да промени името си, фамилията и потребителското име, като се проверява дали нововъведеното име е вече заето
 - `changedUserPassword` – функция, която проверява дали старата парола е вярна, и ако е – записва новата парола, като отново се хешира с `bcrypt`
 - `userProfileData` – функция, която връща цялостната информация за потребителя
- `Admin.js` –
- `getUserCount` – функция, която връща общия брой регистрирани потребители в базата данни.
 - `getTrackCount` – функция, която извлича общия брой качени песни в системата.
 - `getArtistCount` – функция, която връща броя на изпълнителите в базата.
 - `getActiveUserCount` – функция, която връща броя на уникалните потребители, слушали музика през последните 7 дни.
 - `getRecentlyListenedTracks` – функция, която връща последно слушаните 20 песни, включително информация за песен, изпълнител, албум и момент на слушане.
 - `insertTrack` – функция, която добавя нова песен в базата данни с предоставените параметри (заглавие, продължителност, път до аудио файл, албум, изпълнител).
 - `getAlbums` – функция, която извлича албумите на даден изпълнител, сортирани по заглавие.
 - `getArtists` – функция, която връща списък с всички изпълнители, сортирани по име.
 - `insertAlbum` – функция, която добавя нов албум в системата с информация за заглавие, изпълнител, година и изображение.
 - `insertArtist` – функция, която добавя нов изпълнител в базата, включително име и изображение.
 - `insertGenre` – функция, която добавя нов жанр в базата, ако все още не съществува.
 - `getAllTracks` – функция, която връща всички песни с информация за албум, изпълнител и други метаданни.
 - `deleteTrackById` – функция, която изтрива песен и всички свързани записи (напр. в плейлисти, история, любими) по ID.
 - `getArtistById` – функция, която връща данните на изпълнител по зададено ID.

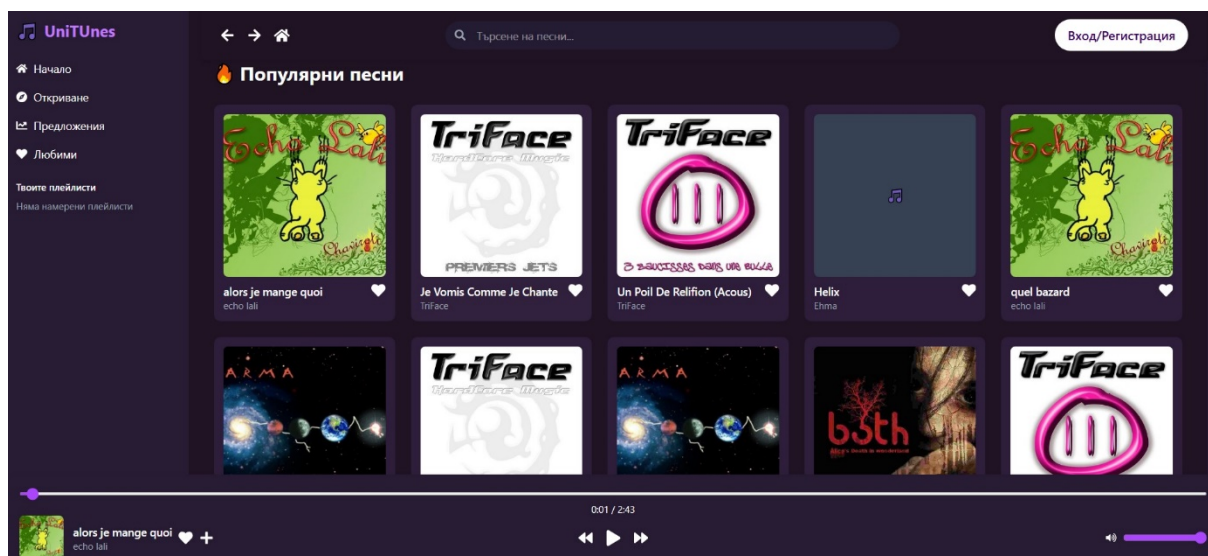
- deleteArtistById – функция, която изтрива изпълнител, неговите песни и албуми, както и всички свързани записи.
 - getAlbumById – функция, която връща данните на албум по зададено ID.
 - deleteAlbumById – функция, която изтрива албум и свързаните с него песни и записи.
 - fetchGenres – функция, която връща списък с всички жанрове, сортирани по азбучен ред.
 - deleteGenreById – функция, която изтрива жанр и премахва връзките му с песни.
 - updateUserRole – функция, която актуализира ролята на потребител на база име на роля.
 - fetchAllUsers – функция, която връща списък с всички потребители, включително роля и дата на регистрация, сортирани по най-нови.
- **Routes** – Пътища - Дефинират се URL endpoint-и и техния http метод (get, post, patch, delete), след което насочват заявката към съответен контролер.
 - index.js
 - albumRoutes.js
 - artistRoutes.js
 - authRoutes.js
 - favoritesRoutes.js
 - friendsRoutes.js
 - musicRoutes.js
 - playlistRoutes.js
 - adminRoutes.js
 - **Utils** – Помощни функции - Съдържа функции, които се използват многократно в приложението – напр. генериране на токени, обработка на дати, валидации и др.
 - authUtils.js – съдържа помощни функции, свързани с проверка на потребителската автентикация и сесия
 - timeUtils.js – приема дата и връща текстов формат като „преди 1 ден“, „преди 1 месец“, „сега“ и т.н. Поддържа секунди, минути, часове, дни, месеци и години. Използва се за да се покаже време на добавяне на песен, редактор и др.
 - **Views** – Изгледи - Съдържа цялата визуална част на приложението, разделена по роля и цел.
 - components – Повторно използвани компоненти като навигация, оформление, форми, бутони и т.н.
 - pages – Основните страници на приложението – например Home, Artist, Album, Playlist и др.

- **Public** – Публични ресурси - Директория, съдържаща всички статични ресурси: CSS, JavaScript, изображения, аудио. Използва се за зареждане от клиента.
 - AlbumImages – това е папката, която съдържа всички изображения на албумите, на които е запазен пътя в базата данни
 - ArtistImages – това е папката, която съдържа всички изображения на изпълнителите, на които е запазен пътя в базата данни
 - Tracks – това е папката, в която се съдържат всички .mp3 файлове, до всеки от които има път в базата данни
 - scrollbarAdjustments.css – този файл дефинира персонализиран външен вид на лентите за превъртане, вместо стандартните, които зависят от браузъра или операционната система. Използва се както за цялата страница, така и за елемент Sidebar от компонентите за повторно използване
- **Temporary** - Временни файлове - Използва се за временни данни или тестови файлове, които не са част от основната логика, но подпомагат разработката или обработката на данни в даден етап.
 - saveTracksAudio.js – Този файл автоматизира процеса по сваляне на аудио файлове, албуми и артисти от Jamendo API и запазването им локално. Той прави няколко неща, като първо взема списък с изпълнители, след това за всеки изпълнител намира неговите албуми и песни. При песни, за които е разрешено сваляне, скриптът сваля .mp3 файла, както и пробва да извлече снимката на албума от аудио мета-данните. Данните се записват в JSON файл, който после може да бъде използван за попълване на база данни. Съхраняват се също локално изображения на изпълнителите и албумите, както и аудио файловете, така че приложението да може да показва визуално съдържание и .mp3 файлове. Скриптът подготвя изход, който може лесно да бъде импортиран в базата данни.
 - generateSQLDataInsertion.js – Файлът взима съдържанието от генерирания JSON файл, и го вмъква в базата данни, като се задават insert заявки, спрямо структурата на базата и внимателно се подготвя всяка таблица – artist, album, track, genre, и trackgenre. Скриптът също така следи за асоцииране на жанрове с песни и създава липсващи жанрове при необходимост.

3. Структура и ~~организация~~организация на потребителския интерфейс

3.1. Начална страница

При влизане в уеб платформата потребителят ще бъде изпратен на началната страница, където той ще разполага с различни функции на употреба на платформата.



Фигура 41 Начална страница на приложението

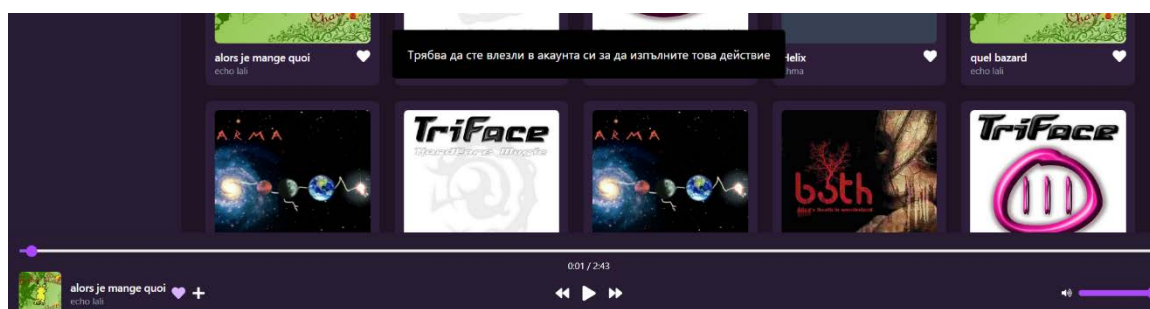
Потреблението на системата ще бъде ограничено, тъй като потребителят все още не е влязъл в системата. Това обаче, няма да му попречи от това да използва уеб приложението и без създаването на потребителски профил.

На началната страница са визуализирани най-популярните песни в платформата базирано на брой слушания, както и най-популярните изпълнители въз основа на броя прегледи на техния профил. Като опашка от песни се задава визуализираните песни в началната страница. Потребителите ще могат да слушат тези песни, чрез натискане върху тях, което ще ги зададе като текуща песен в музикалния плейър. Те ще могат да навигират през опашката от песни чрез използването на бутоните на музикалния плейър, както и да увеличават или намаляват силата на звука на песента.

Потребителите също ще могат да преглеждат профилите на популярните изпълнители, където те ще могат да слушат техните топ песни и да разглеждат албумите им.

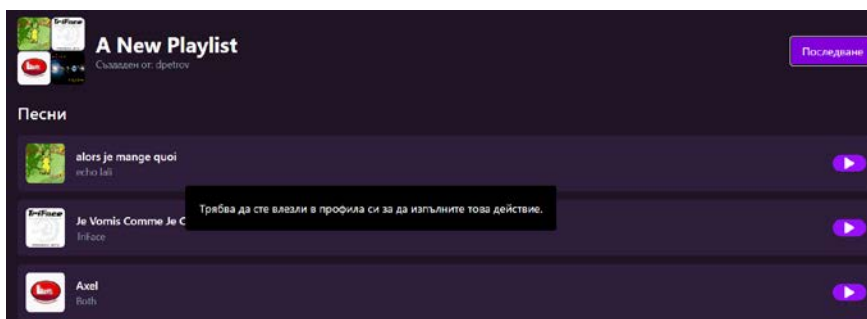
Ограниченията, които са **въведени** за потребителите, които използват системата без създаването на акаунт са:

- Добавяне на песен в любими – Когато потребителят се опита да добави слушаната песен в любими, съответното съобщение ще бъде изписано на екрана, че потребителят няма възможност да добави песента.



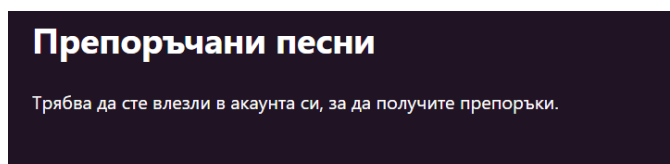
Фигура 42 Опит за добавяне на песен в любими без профил

- Преглед на страница за любими песни – Тъй като потребителят не е влязъл в акаунта си, той няма как да има любими песни, следователно при опит за посещаване на страницата за любими песни, те ще получат съответното съобщение предоставяйки ги с нужната информация.
- Създаване на плейлист/Добавяне на песен в плейлист – Ако потребителят се опита да добави слушаната от него песен в плейлист, съответното съобщение ще бъде отново визуализирано на екрана.
- Последване на плейлист – Когато потребителят се намира на страницата за откриване на музикално съдържание, те ще имат възможността да преглеждат публични плейлисти, създадени от други потребители, както и да слушат песните, които са налични в тези плейлисти. Когато те обаче се опитат да последват конкретен плейлист, съответното съобщение за нужен акаунт ще бъде показано.



Фигура 44 Опит за последване на плейлист без профил

- Последване на изпълнител – Въпреки, че потребителите имат право да преглеждат профилите на изпълнителите, те няма да имат право да ги последват. Следователно, при опит за последване на изпълнител, ще бъде показано съобщение за нужен акаунт.
- Преглед на страница за предложения – Тъй като не се следи, какво слуша потребителят без да е влязъл в системата, при опит той да посети страницата за предложения, музикалното съдържание няма да бъде заредено.



Фигура 45 Опит за преглед на препоръки без профил

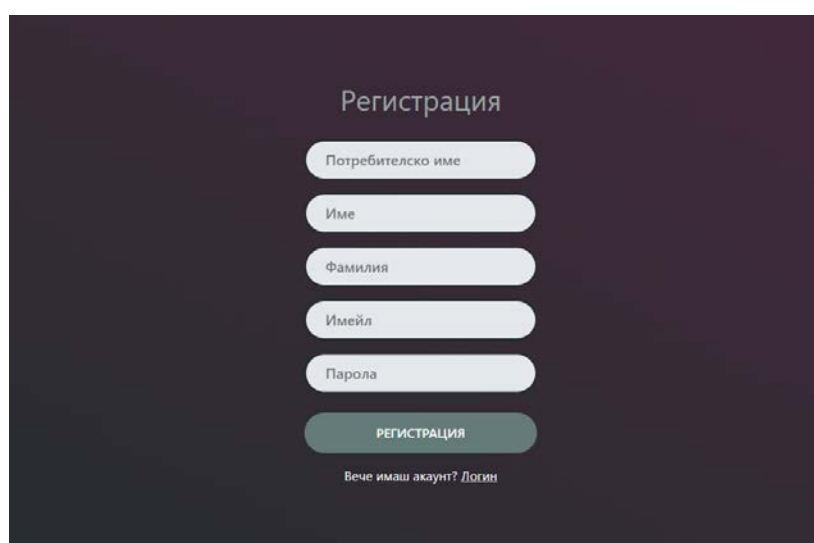
- Преглед на страница за личен профил и приятели – На компонента, който представлява навигационната лента вместо да бъде показан бутон, съдържащ падащо меню с опции за профил и приятели, ще бъде визуализиран бутон за вход или регистрация в системата.

При нуждата на потребителят да се възползва от всички функционалности, които платформата предлага, той ще трябва да си създаде собствен акаунт или да влезе в системата с вече създаден от него акаунт.

3.2. Страница за регистрация и вход

Когато потребителят натисне бутона за вход или регистрация, той ще бъде препратен към страницата за влизане в системата. Той ще разполага с две опции, които са регистриране на нов акаунт или влизане с вече създаден акаунт, в случай, че вече разполага с такъв.

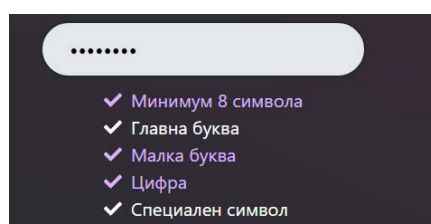
Когато потребителят избере опцията за регистрация, той ще трябва да въведе своите данни за регистрация:



Фигура 46 Страница с форма за регистрация

Въведените данни трябва да спазват предварително зададени ограничения, за да може да се продължи с успешна регистрация:

- Дължината на потребителското име трябва да бъде между 3 и 35 символа
- Потребителското име може да съдържа само цифри, латински букви и долни черти
- Името и фамилията трябва да съдържат само букви
- Паролата трябва да бъде минимум 8 символа, като също трябва да съдържа главна и малка буква, специален символ и цифра, за да може да се постигне максимална сигурност



Фигура 47 Валидация на парола

- Потребителското име и имейл не трябва да съществува в системата
- Всяко едно поле по време на регистрация или вход трябва да бъде попълнено с данни

В случай, че данните, които потребителят е въвел са невалидни, се визуализира съобщение за грешка, след което потребителят трябва да повтори регистрацията въвеждайки правилните данни.

The screenshot shows a registration form with the following elements:

- Title: **Регистрация**
- Username field: Contains "иван\$%". Below it is a message box: "Потребителското име трябва да е между 3 и 35 символа и да съдържа само цифри, латински букви и долни черти."
- First name field: Contains "Иван".
- Email field: Contains "iivanov@gmail.com".
- Password field: Contains masked characters "*****".
- Registration button: Labeled "РЕГИСТРАЦИЯ".
- Footer: "Вече имаш акаунт? [Логин](#)".

Фигура 48 Грешни данни за регистрация

След като потребителят е въвел своите данни правилно и преди да се премине към успешното създаване на профила, паролата, която е въведена се хешира, чрез криптографски алгоритъм bcrypt, за да се максимизира защитата на данните. Bcrypt представлява метод за сигурно преобразуване на пароли чрез хеширане, преди те да бъдат съхранени в базата данни. Вместо паролата да се запази в оригинален вид, тя се кодира по начин, който е практически невъзможен за обръщане на_обратно. Алгоритъмът използва „сол“, което представлява допълнителна случайна стойност, която се комбинира с паролата, така че крайният резултат да бъде уникален дори при еднакви въведени пароли от различни потребители. Освен това, за да се направи brute-force атака по-трудно, Bcrypt прилага серия от повторения на изчисленията, което забавя процеса нарочно. По този начин, ако някой се опита да отгатне парола чрез автоматични методи, ще му отнеме значително повече време. Използването на този подход гарантира по-висока степен на сигурност и защита на потребителските данни при регистрация.

Когато потребителят вече е създал своя акаунт успешно, той може да премине към страницата за вход в системата, където той трябва да въведе своето потребителско име и парола, които е използвал за регистрация:

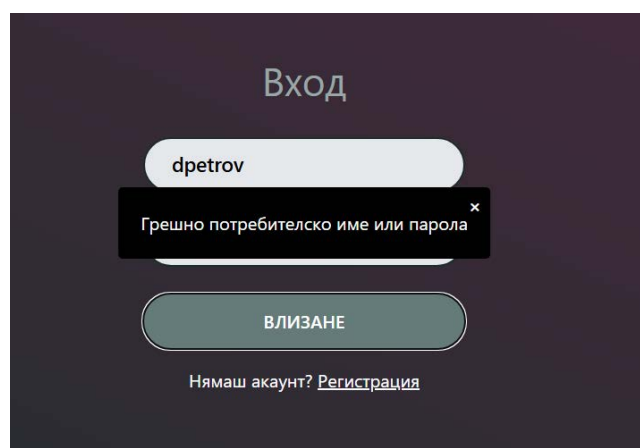


Фигура 49 Страница с форма за вход

След въвеждането на данните се проверява дали потребител с такова потребителско име съществува. Ако е намерен потребител вече хешираната парола в базата данни на този потребител се сравнява с въведената парола отново чрез функция предоставена от bcrypt. Ако паролите съвпадат, се генерира JSON Web Token (JWT), който съдържа основна информация за потребителя, като неговото id, потребителско име и роля. Този токен се записва в бисквитка с ограничен достъп от страна на клиента и срок на валидност от един час. Така се осигурява сигурен достъп до системата и се поддържа сесията на потребителя без нужда от повторно въвеждане на данни.

JWT представлява структуриран и сигурен начин за предаване на информация между сървър и клиент под формата на цифрово подписан токен. Той се състои от три части – заглавна част (header), полезен товар (payload) и подпис (signature). Заглавната част описва типа на токена и използвания алгоритъм за подписване, полезният товар съдържа реалните данни, а подписът гарантира, че съдържанието не е било променено. Благодарение на цифровия подпис, токенът може да бъде валидиран от сървъра без необходимост от съхраняване на сесии в база данни, което го прави ефективен и сигурен механизъм за удостоверяване на самоличност в уеб приложения.

Ако данните, които е въвел потребителят са грешни се визуализира съответното съобщение за грешка:

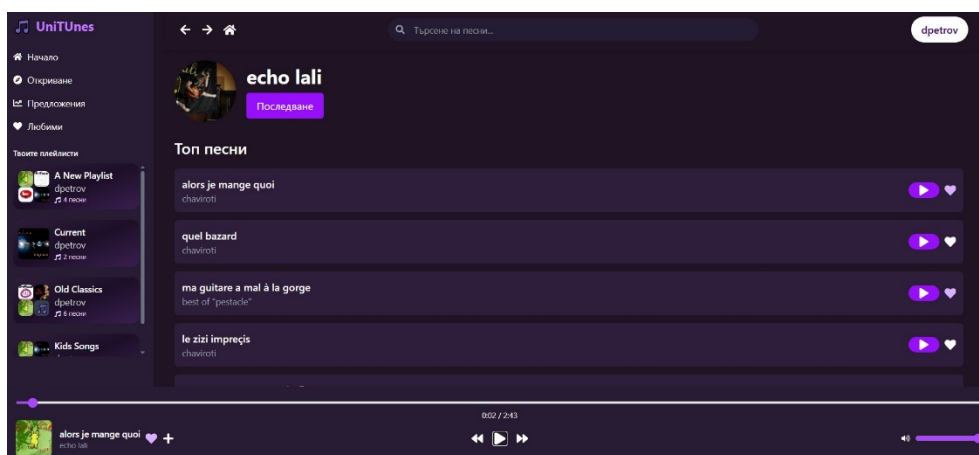


Фигура 50 Грешни данни за вход

След записването на токена и успешната автентикация, въз основа на ролята на потребителя се препраща към началната страница на системата, ако неговата роля е на обикновен потребител и към администраторския панел, ако ролята му е на обикновен или главен администратор.

3.3. Страници за изпълнители и албуми

При отваряне на профил на изпълнител, потребителят ще може да види налична за него информация като неговите най-популярни песни, въз основа на брой слушания и всички налични албуми, които конкретния изпълнител е издал:



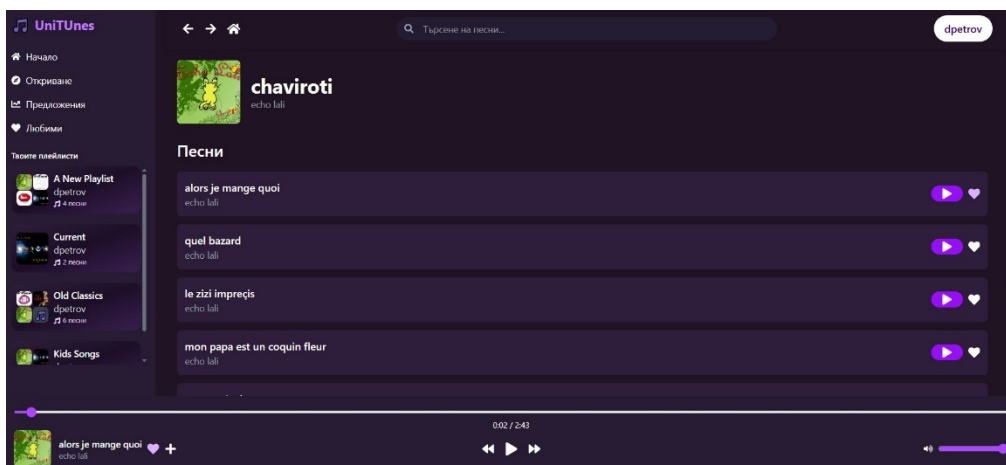
Фигура 52 Страница за изпълнител



Фигура 51 Налични албуми на изпълнител

Потребителите ще имат възможността да последват профила на даден изпълнител, който после ще могат да достъпят от техния профил. Опашката от песни се задава на най-популярните песни на изпълнителя при превключване на неговия профил, като потребителите могат да добавят всяка една от песните в профила му в любими. При натискане на бутона за пускане на песента, песента се задава като текуща в музикалния плейър, увеличават се слушанията на песента с едно, добавя се в историята със слушани песни на текущия потребител и съответния бутон се сменя, както в плейъра така и на самата страница на изпълнителя, за да се избегне конфликт с пускането на песните и за по-лесна визуализация за самия потребител.

При натискане на някой от визуализираните албуми, потребителят ще бъде изпратен на страницата на конкретния албум:

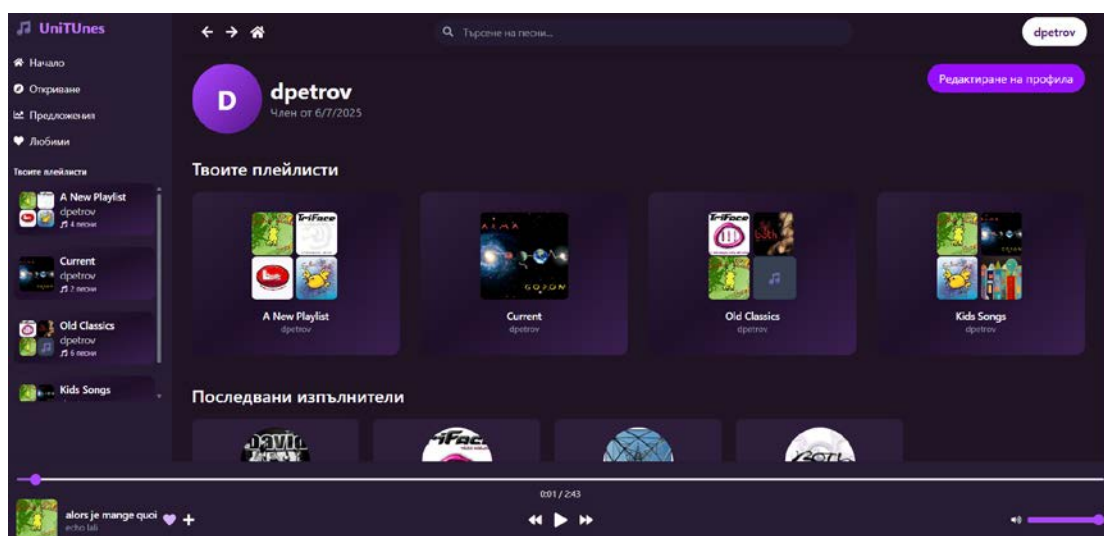


Фигура 53 Страница за албум

В страницата с конкретния, избран албум от потребителя се визуализират всички песни, които са част от този албум, които потребителя може да слуша. При превключване на страницата с албума опашката от песни се задава на песните от самия албум, без да прекъсват слушаната от потребителя песен в текущия момент. Потребителят отново разполага с опцията да добави всяка от песните в албума в любими, както от самия албум, така и от музикалния плейър. При натискане на бутона за пускане на песента, песента отново се задава като текуща в музикалния плейър, увеличават се слушанията, добавя се в историята със слушани песни и съответния бутон се сменя, както в плейъра така и на самата страница на албума.

3.4. Страница за профил

Чрез натискане върху бутона за профила си в навигационната лента потребителите могат да достъпят своя личен профил:

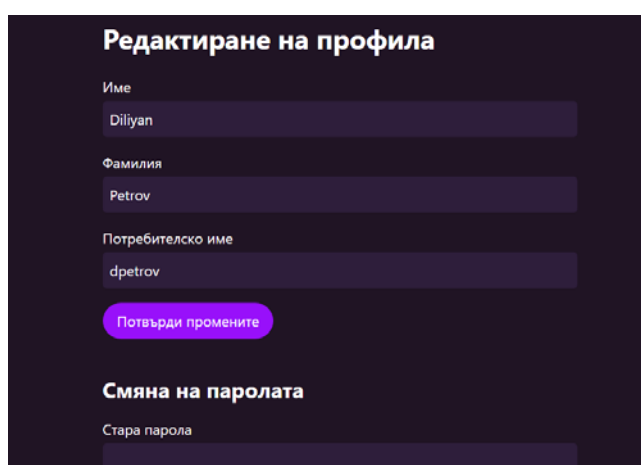


Фигура 54 Страница за профил на потребителя

Когато потребителят влезе в страницата на профила им те могат да видят датата на тяхната регистрация в системата. Под името и датата на регистрация се визуализират всички плейлисти, които потребителят е създал, следва или на които е ~~редактор~~ редактор. При натискане върху тях, той ще бъде препратен към страницата на конкретния плейлист.

Под плейлистите са визуализирани всички изпълнители, които потребителят следва. Отново чрез натискане върху тях, той ще бъде изпратен в техния профил. Тази функционалност предоставя лесен начин на потребителите да следят харесваните от тях изпълнители в своя профил.

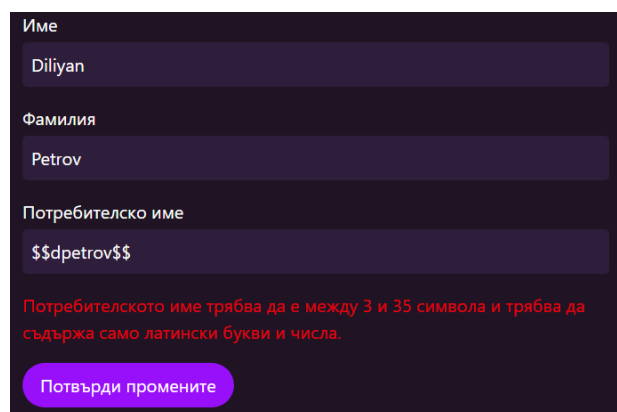
При натискане на бутона за редактиране на профила, потребителят ще бъде изпратен към страница, където той може да редактира данните за своя профил:



Фигура 55 Страница за редактиране на профила

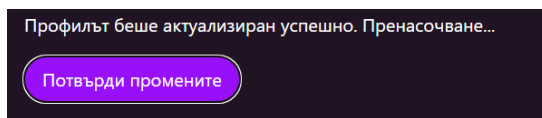
Потребителят може да редактира своето име, фамилия, потребителско име и парола. При въвеждане на данните за промяна на името, фамилията или потребителското име се прави валидация спрямо дефинираните правила, като името може да съдържа само букви и потребителското име може да съдържа само латински букви и цифри без специални символи и др.

Ако въведените данни от потребителя са неправилни, се извежда съответното съобщение за грешка под полетата за въвеждане:



Фигура 56 Грешни данни за актуализация на профила

При правилно въведени данни, ~~профила~~ профилът на потребителя се актуализира успешно, след което се изписва съответното съобщение за успешното запазване на промените и потребителят се пренасочва към главната страница на



Фигура 57 Съобщение за успешна актуализация на профила

профила му.

Втората опция за актуализация на профила е чрез смяна на паролата на потребителя. Той трябва да въведе своята стара парола, след което трябва да въведе нова парола. Отново е включена визуализацията на критериите за парола, за да предостави яснота на потребителите за правилата, които трябва да се спазят при задаване на нова парола. След като данните са въведени паролата, която е въведена се сравнява с хешираната парола в базата данни на текущия потребител, чрез функция на bcrypt.

Ако данните въведени от потребителя не са правилни се изписват съответните съобщения за грешка:

- При несъвпадаща стара парола
- При несъвпадаща нова парола в полето за ~~потвърждение~~ потвърждение на паролата
- При парола която не отговаря на изискванията

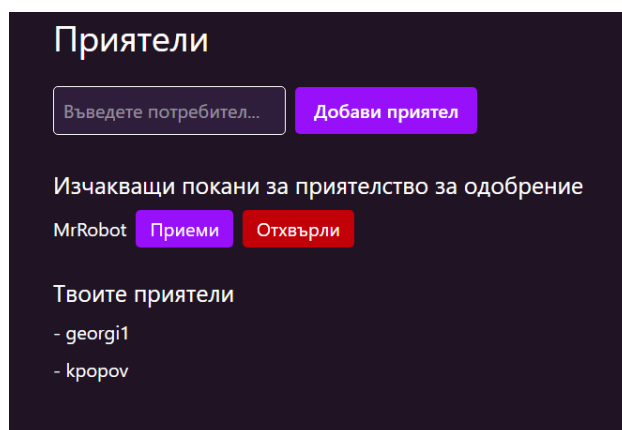
Изображение на формуляр за смяна на паролата с заглавие "Смяна на паролата". Формулярът е разделен на секции: "Стара парола" (с поле за въвеждане), "Нова парола" (с поле за въвеждане), списък от четири проверени критерия (Минимум 8 символа, Главна буква, Малка буква, Цифра, Специален символ), "Потвърждение на новата парола" (с поле за въвеждане) и грешка: "Паролата не отговаря на изискванията." в червен цвят. В долната част има син бутон "Промяна на паролата".

Фигура 58 Грешни данни за смяна на парола

При правилно въведени данни системата хешира новата парола чрез bcrypt и я задава в базата данни и след това се изписва съответното съобщение за успешната смяна на паролата и потребителя е пренасочен към началната страница на своя профил.

3.5. Страница за приятели

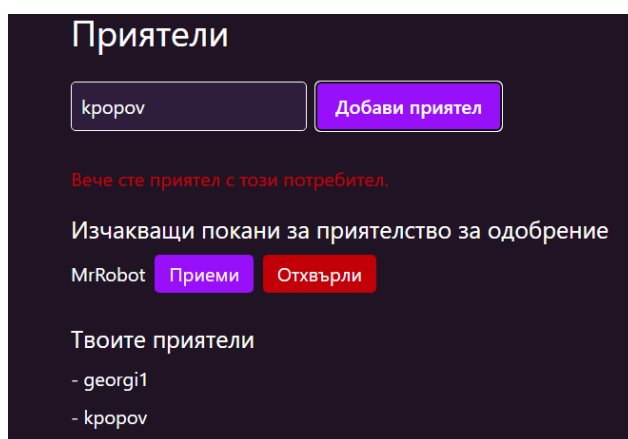
След натискане на опция Приятели от падащото меню на бутона за профил, потребителят се препраща към страницата за приятели:



Фигура 59 Страница за приятели

Показват се всички налични опции за приятели, като най-горе потребителите имат възможност да изпратят покана за приятелство на друг потребител, чрез въвеждане на неговото потребителско име.

Ако към потребителя вече има изпратена заявка, изпращачът вече е приятел с потребителя или потребителското име съвпада с това на изпращача се извежда съответното съобщение за неуспешното изпращане на покана за приятелство:



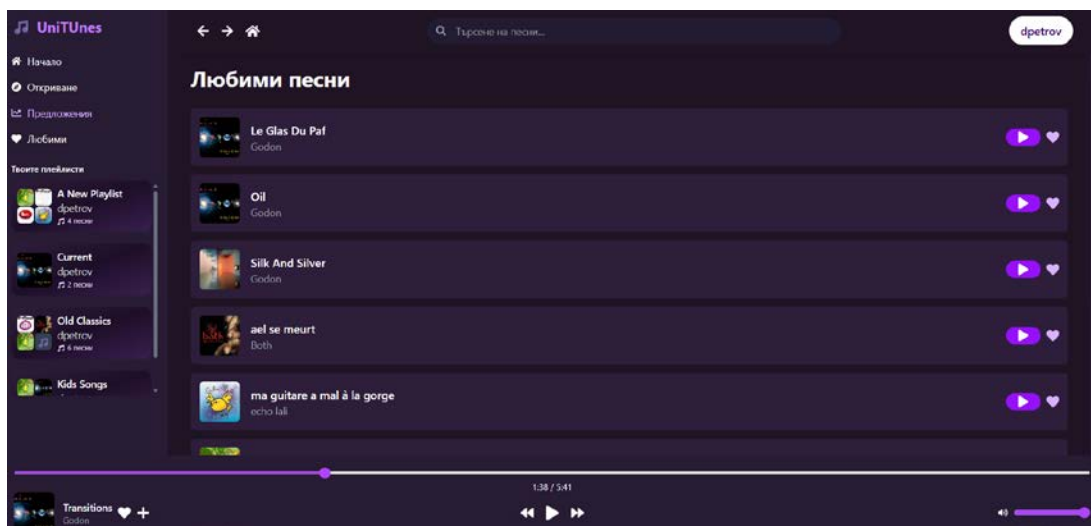
Фигура 60 Изпращане на покана на приятел

При успешно изпращане на поканата се извежда съответното съобщение. Под опцията за изпращане на покана се намират и изпратените към потребителя покани за приятелство. Приемане на покана ще доведе до добавяне на приятелството на потребителите в базата данни, а при отказ поканата ще се маркира като отказана и

потребителите няма да станат приятели. Под поканите се намира списък с всички налични приятели, които текущият потребител има.

3.6. Страница за любими песни

Когато потребителят натисне върху бутона Любими, който се намира на страничната лента, той ще бъде препратен към страницата с любими песни, където той може да види всички песни, които е добавил:

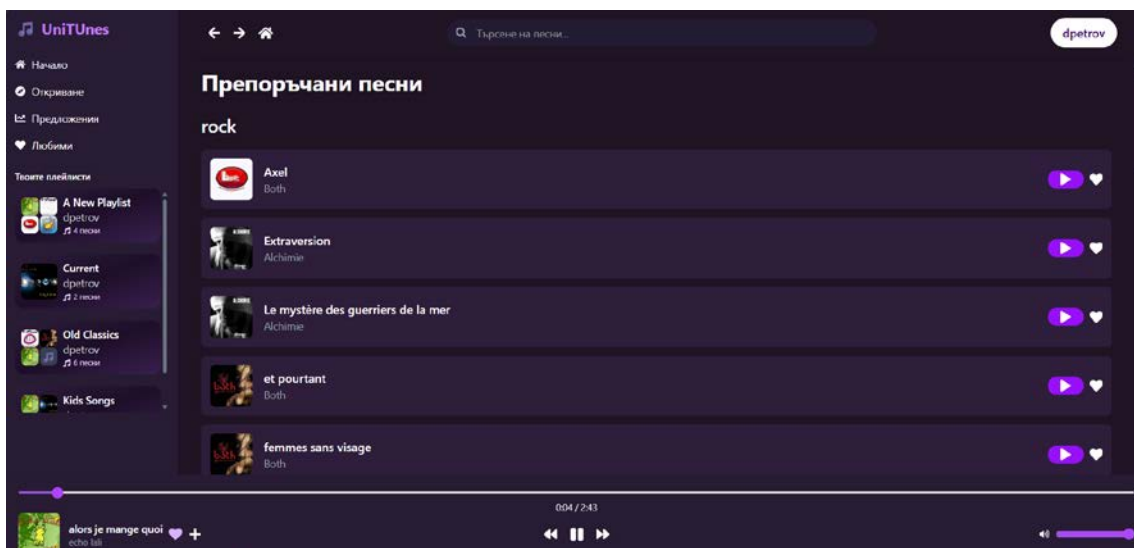


Фигура 61 Страница за любими песни

Опашката с песни се задава на всичките харесани песни на потребителя. При натискане на някой от бутоните за добавяне в любими на песните, тя ще бъде премахната от списъка и съдържанието на любимите песни ще бъде автоматично актуализирано, като тази песен ще бъде премахната от опашката на песните. При натискане на бутона за пускане на песента, песента се задава като текуща в музикалния плейър, увеличават се слушанията, добавя се в историята със слушани песни и преминава в режим на слушане като бутоните отново се променят в и плейъра и страницата с любимите песни.

3.6. Страница за предложения

Когато потребителят натисне върху бутона Предложения, който се намира на страничната лента, той ще бъде препратен към страницата с препоръки, където той може да види всички песни, които са му препоръчани:

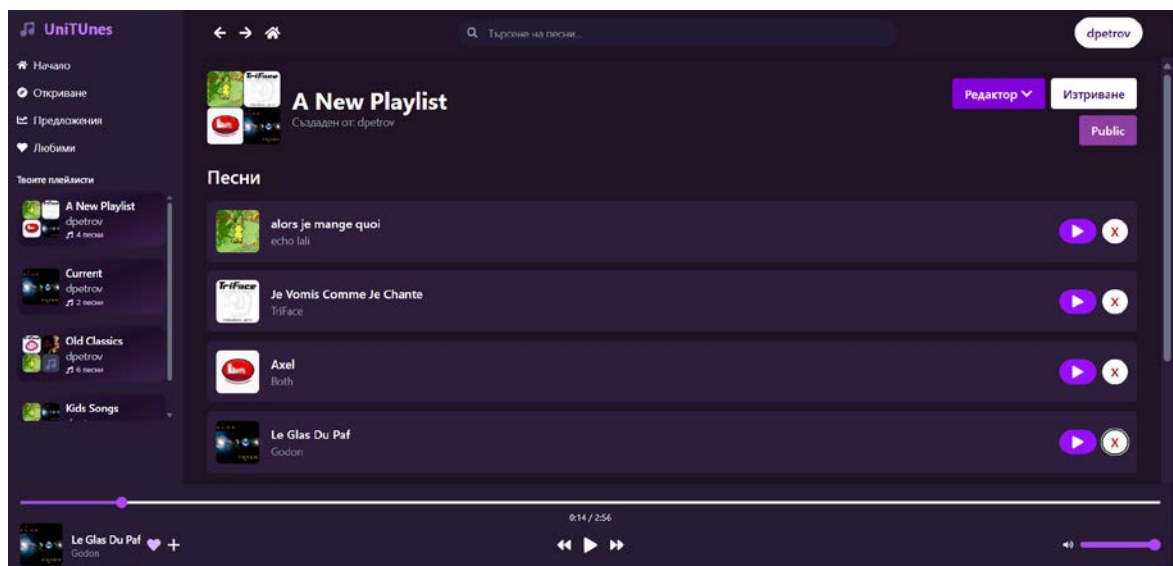


Фигура 62 Страница за предложения

Песните са групирани по жанрове, като всяка група се визуализира отделно, с възможност за преминаване между страниците, като групирането става по 10 песни на страница. При натискане на бутона за пускане, песента се задава като текуща и започва да се възпроизвежда в музикалния плейър, а също така се отчита и ново слушане в системата. Именно чрез това отчитане на слушане се формира и списъкът с препоръчани песни – системата автоматично проследява какви песни слуша потребителят и кои жанрове са му най-интересни през последните 30 дни. На база на това поведение, се избират най-често слушаните от него жанрове и спрямо тях системата търси и подбира нови, интересни предложения в същите жанрове. Тези песни се извличат случайно, за да може всяко влизане в страницата да предлага нещо ново и различно. При повторно натискане на същия бутон за пускане се превключва между пауза и възпроизвеждане. Потребителят отново има възможност да добавя харесана от него песен в любими чрез съответния бутон до бутона за възпроизвеждане. Целият списък с препоръчани песни се използва и за задаване на текущата опашка на музикалния плейър, като по този начин потребителят може лесно да прослуша всички предложения едно след друго.

3.7. Страница за плейлисти

При натискане върху някой от плейлистите в страничната лента или в личния профил на потребителя, те могат да навигират до страницата на самия плейлист, където ще са визуализирани песните, които са част от плейлиста:

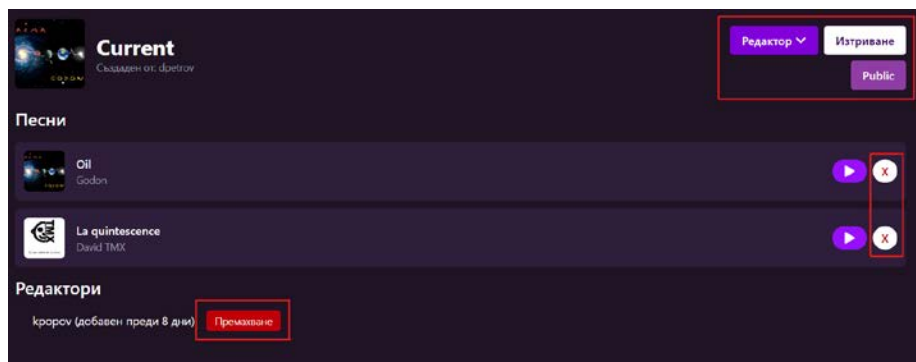


Фигура 63 Страница за плейлисти

В най-горната част от плейлиста се визуализира информация за името на плейлиста, неговия създател, както и изображение на плейлиста, което е композирано от изображенията на първите четири песни на плейлиста. Когато плейлистът е създаден се задава начално изображение без визуализирани песни. Когато потребителят е добавил от 1 до 3 песни в плейлиста, визуализирана ще бъде само снимката на първата песен като изображение на плейлиста. Тази функционалност се реализира както и в страницата на самия плейлист, така и в страничната лента.

Въз основа на каква е ролята на потребителя в този плейлист, ще бъдат визуализирани различни бутони за взаимодействие със съдържанието на плейлиста:

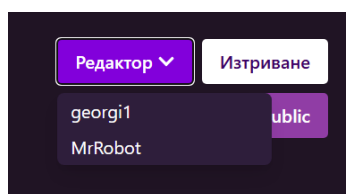
- Ако потребителят е създател на плейлиста ще бъдат визуализирани следните бутони:



Фигура 64 Собственик на плейлист

Създателят на плейлиста разполага с пълните правомощия на взаимодействие с плейлиста, както и да дава правомощия на други потребители. Той може да добавя и премахва песни от плейлиста. Премахването става като натисне бутона разположен до бутона за възпроизвеждане. При създаване на плейлист автоматичния статус на плейлиста, който се задава е частен. Създателят ще има възможността да смени този статус на публичен. Като разликата е, че когато плейлиста е частен, той няма да бъде видим за други потребители освен създателя, дори и вече да има потребители, които да го следват или са негови редактори, а като е публичен той ще е достъпен за всички потребители да го последват, както редактори да добавят и премахват песни.

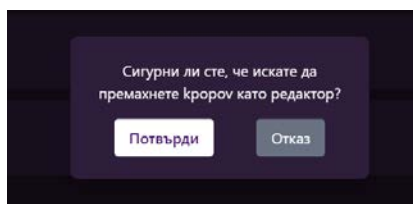
При избиране на бутон Редактор, създателят може да добави някой от своите приятели, като им даде правомощия да редактират плейлиста:



Фигура 65 Списък с редактори

След като създателят избере някой от предложените редактори, той се добавя в плейлиста като редактор, страницата се актуализира и се обновява списъка с редактори като се визуализира нововъведения редактор. Когато създателят на плейлиста избере бутон Премахване на някой от редакторите е последван от съобщение за потвърждение дали иска да премахне потребителя като редактор:

След като е избрана опцията Потвърди, редактора е премахнат от плейлиста, както и



Фигура 66 Потвърждение за премахване на редактор

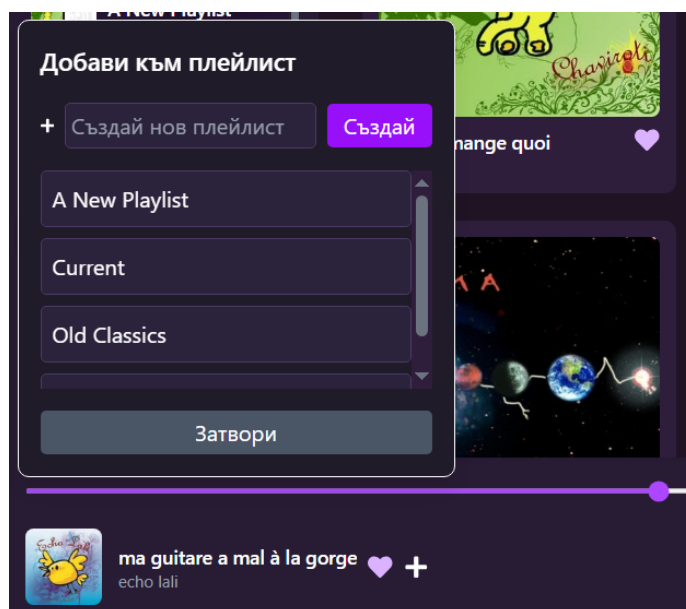
от списъка с редактори.

В случай, че потребителят иска да изтрие плейлиста, той може да го направи чрез съответния бутон, след което той отново е длъжен да потвърди чрез прозорец за потвърждение. След като се потвърди изтриването плейлиста се премахва от базата, като се изтриват всички последователи, редактори и песни, част от него.

- Ако потребителят е редактор на плейлиста ще бъдат визуализирани
- Ако потребителят е последовател на плейлиста или просто го преглежда ще бъде показана единствено опцията за последване на плейлиста.

При зареждане на страницата с плейлиста, опашката с песни се задава да е на песните, които са част от плейлиста. Когато потребителят избере бутона за възпроизвеждане песента се задава като текуща в музикалния плейър, увеличават се слушанията на песента и тя се добавя в историята със слушани песни.

Добавянето на песни в плейлист става чрез опцията „+“ в плейъра:

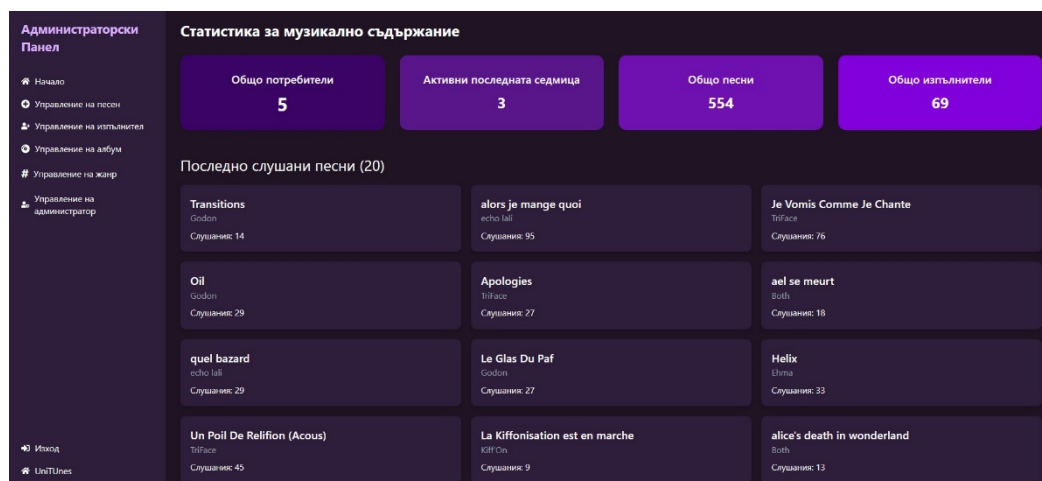


Фигура 67 Добавяне на песен в плейлист

След като опцията е селектирана се отваря контекстно меню, което показва списък с плейлистите, които потребителят е създал или има права да редактира, като същевременно дава възможност да се създаде и нов плейлист. Потребителят въвежда името на плейлиста за създаване, след което след натискане на бутона Създай, плейлиста се създава в базата данни и списъкът с плейлисти се актуализира, като новия плейлист е една от опциите. При избиране на плейлист от този списък за добавяне на песен, избраната песен се добавя към съответния плейлист чрез заявка към бекенда, след което интерфейсът се обновява, за да отрази промените. След успешното добавяне, прозорецът се затваря автоматично. По този начин се улеснява управлението на музикалното съдържание и организирането му по предпочитания на потребителя.

3.8. Страница за администратор

Когато потребителят се автентикира като администратор по време на процеса на влизане в системата той ще бъде изпратен в началната страница на администраторския панел:



Фигура 68 Страница за администратор

На началната страница на администраторския панел са разположени главните функционалности, които предлага панелът, като от лявата страна се намира страничната лента с навигационни бутони за опериране на данни за музикално съдържание, както и на потребители.

В главната част на страницата се намират общи статистики за потреблението на уеб платформата, както и данни за музикалното съдържание:

- Общ брой потребители, които имат създаден акаунт
- Потребители, които са използвали платформата в последните 7 дни
- Общ брой песни и изпълнители налични в системата

Под тези статистики са визуализирани последните 20 слушани песни в приложението, като е показан тяхното име, изпълнител и колко общо брой пъти са слушани дадените песни.

В страничната лента главната част от опциите се използват за управление на музикално съдържание:

- Управление на песен – Добавянето на песен става като се въведе заглавието на песента, след което се избере валиден .mp3 файл. След като се избере файл дължината на песента се изчислява автоматично и се попълва също автоматично. Избира се изпълнител от падащо меню, като също има опция да се търси, като се въведе неговото име. След като се въведе изпълнител долното падащо меню визуализира всички албуми, които този изпълнител притежава, от който администраторът може да селектира. Втората опция за управление на песните е изтриване на песен, като от падащо меню и поле за търсене се селектира съответната песен и чрез бутон за изтриване администраторът потвърждава изтриването в съответния прозорец.

Добави нова песен

Заглавие на песента

Song #3

Качи .mp3 файл

Choose File track_199.mp3

Продължителност (секунди)

170

Изпълнител

 Acoustic Affinités [Смени изпълнителя](#)

Албум

Dans quel cirque je vis

[Добави песен](#)

Изтриване на песен

Търси песен...

1664 (Rouler Pinder) [Изтрий](#)

Фигура 70 Управление на песен

Добави нов изпълнител

Име на изпълнителя

Качи изображение

Choose File No file chosen

[Добави изпълнител](#)

Изтриване на изпълнител

Търси изпълнител...

 Acoustic Affinités [Изтрий](#)

Фигура 69 Управление на изпълнител

- Управление на изпълнител – Добавянето на изпълнител се

осъществява като се въведе неговото име и се избере изображение за снимка на профила. След като данните са валидирани и се премине към добавяне на изпълнителя, изображението, което е избрано, също се добавя в папката с останалите изображения на изпълнители и в системата се записва пътя до новозапазения файл. Опцията за изтриване се реализира като се избере изпълнител и отново чрез прозорец за потвърждение администраторът го изтрива от системата заедно с всички негови албуми и песни. При изтриване на изпълнителя заедно с него се изтрива и запазеното изображения в папката с изображения на изпълнителите.

- Управление на албум – Добавянето на албум става чрез въвеждане на заглавие, дата на издаване, качване на снимка на албума и изпълнител, който е издал албума. Изтриването става като се селектира изпълнител от падащо меню, след което се визуализира второ падащо меню с всички албуми издадени от този изпълнител. След като се избере албум от падащото меню, чрез бутон за изтриване администраторът може да изтрие албума, заедно с всички песни,

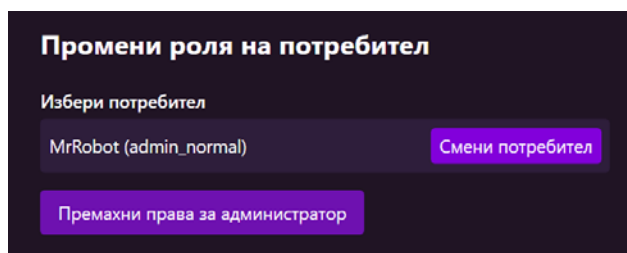
Фигура 71 Управление на албум

които са част от него.

- Управление на жанр – Добавянето на жанр се реализира чрез

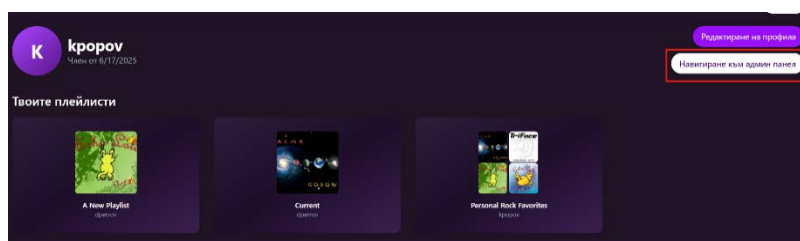
въвеждане на името на жанра. При опит за добавяне се проверява дали такъв наличен жанр съществува вече в системата и ако такъв съществува заявката се отхвърля и се изписва съответното съобщение. Изтриването на жанр става като се избере конкретен от падащото меню и с потвърждение в съответния прозорец. Изтрива се жанрът от системата както и от песните, към които е бил начислен.

- Управление на администратор – Функционалностите за администратора са достъпни само за главния администратор и обикновените администратори няма да виждат тази опция. Тя се реализира като се селектира потребител от системата, срещу който се изписва неговата роля. В зависимост от ролята на потребителя, който е избран, се визуализира бутон за добавяне или премахване на администраторски права. При опит за добавяне на администраторски права на потребител от главния администратор, който вече има такива се изписва съответното съобщение.



Фигура 73 Управление на администратор

Администраторите също имат опцията да използват музикалната платформа, като имат възможност да навигират до нея чрез натискане на бутон UniTunes. Това им позволява също да се възползват от функционалностите, които предлага системата, което ще повиши тяхната запознатост с нея, както и тестване на направени от тях проблеми и следене за проблеми. За да навигират обратно към администраторския панел те могат да отворят своя профил и чрез бутон, видим само за администраторите, да се върнат обратно.

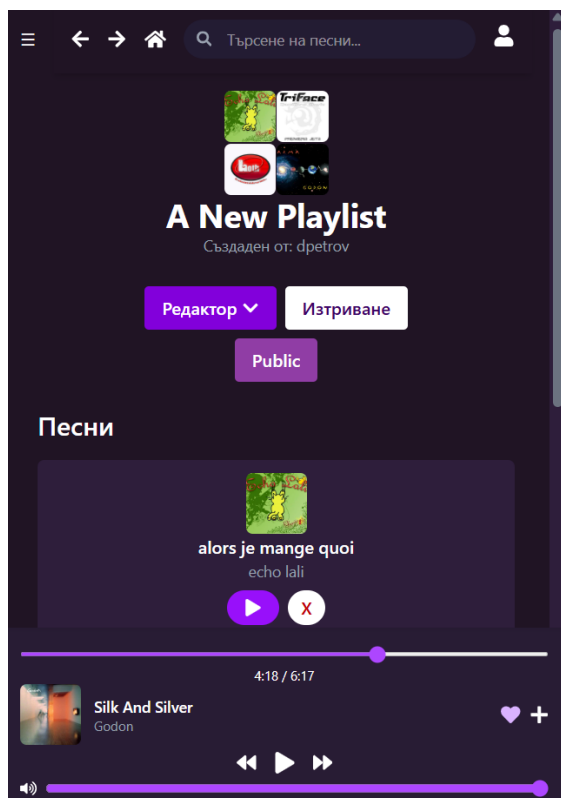


Фигура 74 Навигация към администраторски панел

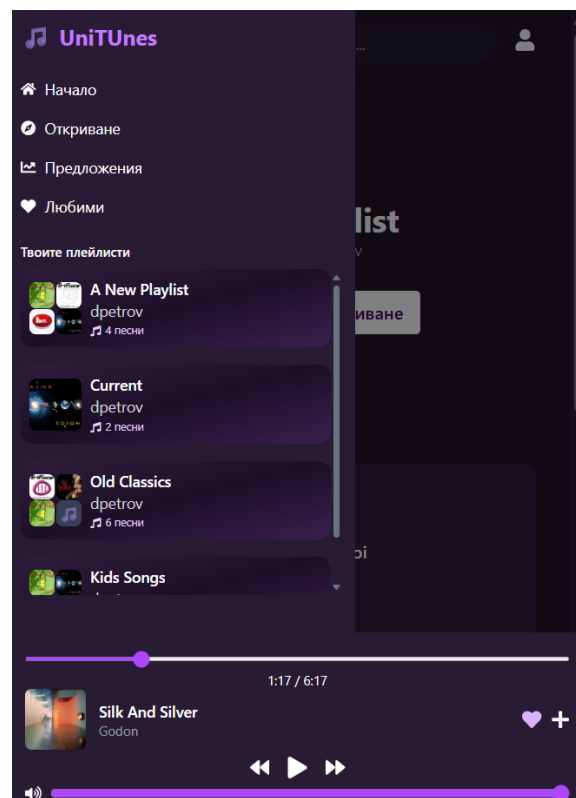
4. Responsive дизайн

За да бъде едно уеб приложение удобно и достъпно за потребителите независимо от устройството, от което се използва, е необходимо дизайнът му да се адаптира динамично спрямо различните резолюции и формати на екрана. При изграждането на интерфейса е взет предвид принципът на responsive дизайн, който осигурява адаптивно поведение на елементите и гарантира, че визуалното оформление и функционалностите се запазват както на мобилни устройства, така и на по-големи екрани като лаптопи и настолни компютри. Това позволява на всеки потребител да работи с приложението интуитивно и без затруднения, независимо от контекста на ползване. Музикалният плейър, песните, албумите, плейлистите, интерактивните компоненти, бутоните и др. променят подредбата и размера си така, че да отговарят на екрана, без да нарушават логиката и последователността на интерфейса, същевременно правейки го да изглежда приятно и лесно за употреба.

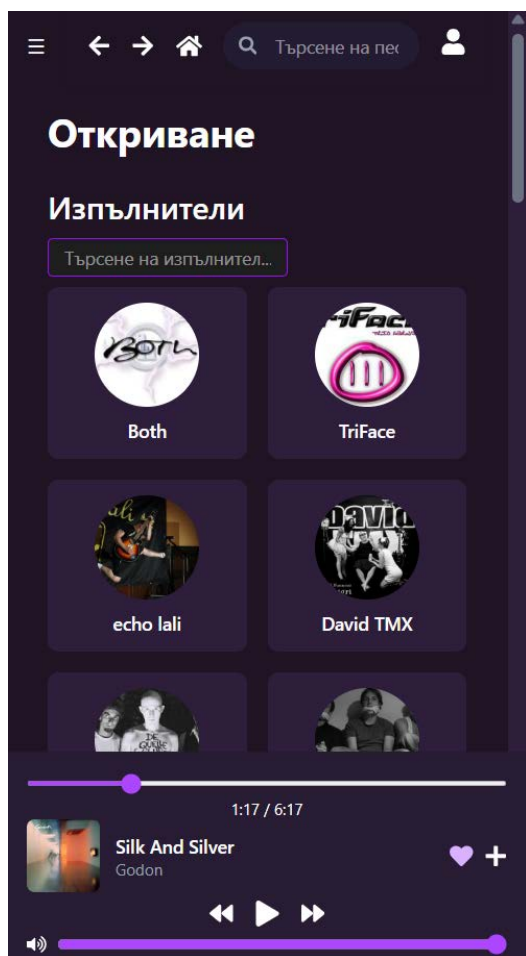
Responsive дизайнът е приложен както в главната потребителска част на системата, така и в администраторския панел.



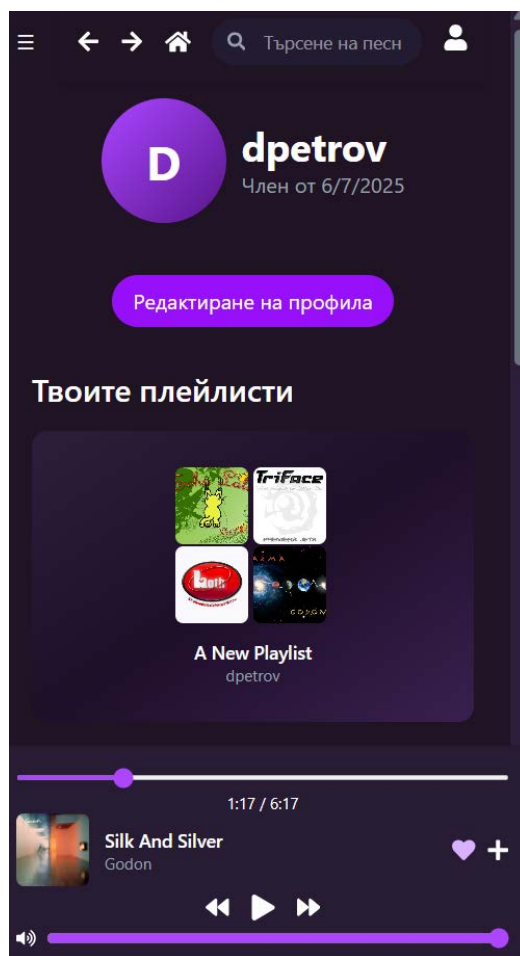
Фигура 75 Responsive дизайн на плейлист страница и на музикален плейър



Фигура 76 Responsive дизайн на Sidebar



Фигура 77 Responsive дизайн на страница откриване



Фигура 78 Responsive дизайн на страница профил

Глава V – Тестове и резултати

Глава VI – Заключение